

國立臺中教育大學資訊工程學系碩士班

碩士論文

指導教授：黃國展 博士

兼顧工作負載和資源規模之
工作流程排程方法研究
Workload- and Resource-Aware
List-Based Workflow Scheduling

研究生：陳冠甫 撰

中華民國 108 年 7 月

July, 2019

摘要

workflow 排程是一個 NP 完全問題，並且也是平行工作排程中重要的研究議題。因此已發展出許多不同類型的演算法來解決這個具有挑戰性的排程問題。而基於列表的工作流排程是其中最被廣泛使用的類型之一，已被成功應用至不同工作流結構和優化目標。在各種基於列表的工作流演算法中，HEFT 和 PEFT 是兩個具有重要創新和眾所周知的代表。但是，美中不足的是它們都未能在所有工作流屬性與系統規模下，始終保持優勢。這是我們進行本論文中研究工作的主要動機之一。我們專注在同構平行系統上的工作流排程問題研究，此一架構在當前廣泛採用虛擬化技術的雲端計算平台上已經變得可行和普遍。我們首先針對 HEFT 和 PEFT 進行了各種情況下的完整實驗分析與比較，發現了一些它們的缺點與原因，然後提出了一種新的兼顧工作負載和資源考量的工作流排程演算法。此一排程方法具有三種新機制，包括考量工作流結構之工作排序方法、著重縮短資源配置後實際關鍵路徑長度之動態工作排序作法、以及具適應性的工作配置機制。我們分別採用隨機合成之工作流程結構和真實世界實際應用中之工作流程模型，進行了一系列的模擬實驗來評估所提出的新排程方法，並將其與 HEFT 和 PEFT 進行了分析比較。實驗結果表明在大多數情況下，我們的方法在工作流的平均完成時間和最佳效能比率方面都優於 HEFT 及 PEFT，獲得了顯著的效能提升。

關鍵字：工作流排程、列表排程、工作排序、工作配置、資源配置後之實際關鍵路徑

Abstract

Workflow scheduling is an NP-complete problem, and has always been an important research topic on parallel job scheduling. Different types of scheduling heuristics have been developed for tackling the challenging workflow scheduling problem. List-based scheduling is one of the most widely used categories for different workflow structures and optimization goals. Among various list-based workflow scheduling algorithms, HEFT and PEFT are two typical and well-known representatives with important innovations. However, none of them can consistently retain advantage over the other across various workflow properties and system scales. That motivated our research work in this thesis, focusing on the problem of task-parallel workflow scheduling on homogeneous parallel systems which have become feasible and common on current cloud computing platforms with virtualization technology. Based on thorough experimental analysis of HEFT and PEFT, we found drawbacks of them, and then developed a new workload- and resource-aware list-based workflow scheduling approach featuring three new mechanisms, including structural task ranking, task ranking based on allocated critical path, and adaptable task allocation. The proposed approach was evaluated with a series of simulation experiments based on both synthetic workflow structures and real-world workflow models, and compared to HEFT and PEFT. The experimental results demonstrate that our approach can achieve significantly superior performance in most circumstances in terms of both average makespan and best ratio.

Keywords: Workflow computing, List scheduling, Task ranking, Task allocation, Allocated critical path

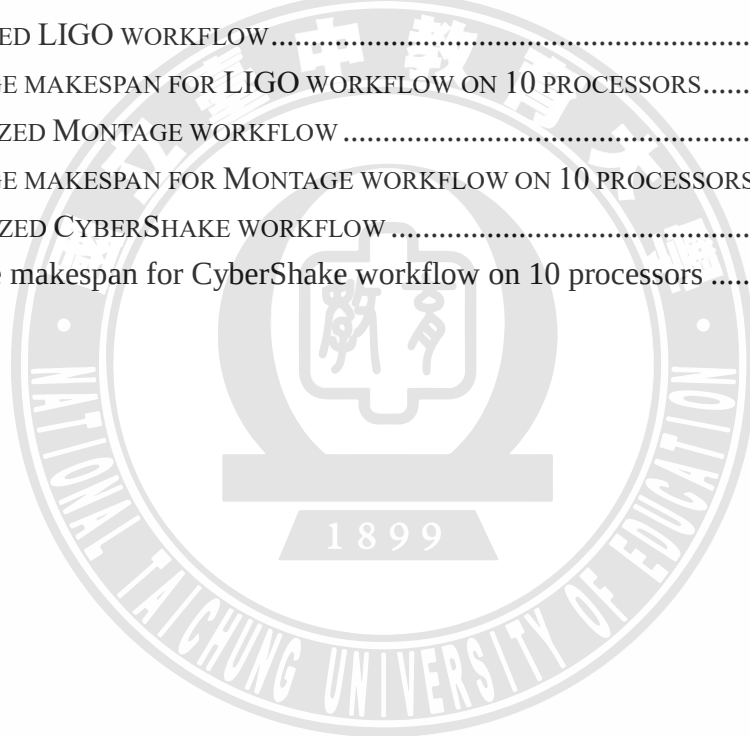
Table of Contents

摘要.....	I
Abstract.....	II
Table of Contents.....	III
List of Figures	IV
List of Tables	VI
Chapter 1. Introduction	1
Chapter 2. Related Work.....	5
2.1 Heterogeneous Earliest Finish Time (HEFT).....	6
2.2 A Lookahead Variant of HEFT	7
2.3 Predict Earliest Finish Time (PEFT).....	8
Chapter 3. List-Based Workflow Scheduling	9
3.1 Comparative Analysis of HEFT and PEFT.....	9
3.1.1 CCR equal to 0.1	15
3.1.2 CCR equal to 1	17
3.1.3 CCR equal to 10	20
3.2 Workload- and Resource-Aware List Scheduling Approach.....	23
3.2.1 Structural Task Ranking	23
3.2.2 Allocated Top Rank.....	28
3.2.3 Adaptable Task Allocation	32
3.2.4 The Entire Workload- and Resource-Aware List Scheduling Approach	35
Chapter 4. Performance Evaluation	38
4.1 Experiments with Synthetic Fork-Join Workflows.....	38
4.2 Experiments with Real-World Workflow Models	43
4.2.1 LIGO.....	43
4.2.2 Montage	44
4.2.3 CyberShake.....	46
Chapter 5. Conclusion.....	47
References.....	49

List of Figures

1.1 WORKFLOW STRUCTURE REPRESENTED BY A DAG	1
3.1 AVERAGE MAKESPAN FOR DIFFERENT PROCESSORS	10
3.2 BEST RATIO FOR DIFFERENT PROCESSORS	11
3.3 AVERAGE MAKESPAN FOR DIFFERENT CCR VALUES	12
3.4 BEST RATIO FOR DIFFERENT CCR VALUES	12
3.5 AVERAGE MAKESPAN WITH 5 PROCESSORS FOR DIFFERENT CCR VALUE	13
3.6 BEST RATIO WITH 5 PROCESSORS FOR DIFFERENT CCR VALUE	13
3.7 AVERAGE MAKESPAN WITH 5 PROCESSORS FOR DIFFERENT BRANCHES	14
3.8 BEST RATIO WITH 5 PROCESSORS FOR DIFFERENT BRANCHES	14
3.9 CCR 0.1 WORKFLOW SCHEDULING RANKING PHASE	15
3.10 CCR 0.1 WORKFLOW ALLOCATION PHASE ON 3 PROCESSORS	16
3.11 CCR 0.1 WORKFLOW SCHEDULING RANKING PHASE	17
3.12 CCR 0.1 WORKFLOW ALLOCATION PHASE ON 10 PROCESSORS	17
3.13 CCR 1 WORKFLOW SCHEDULING RANKING PHASE	18
3.14 CCR 1 WORKFLOW ALLOCATION PHASE ON 3 PROCESSORS	18
3.15 CCR 1 WORKFLOW SCHEDULING RANKING PHASE	19
3.16 CCR 1 WORKFLOW ALLOCATION PHASE ON 10 PROCESSORS	19
3.17 CCR 10 WORKFLOW SCHEDULING RANKING PHASE	20
3.18 CCR 10 WORKFLOW ALLOCATION PHASE ON 3 PROCESSORS	21
3.19 CCR 10 WORKFLOW SCHEDULING RANKING PHASE	22
3.20 CCR 10 WORKFLOW ALLOCATION PHASE ON 10 PROCESSORS	22
3.21 COMMON INTER-TASK RELATIONSHIP IN WORKFLOW STRUCTURE	24
3.22 CCR 0.1 WORKFLOW SCHEDULING RANKING PHASE	26
3.23 CCR 0.1 WORKFLOW ALLOCATION PHASE ON 3 PROCESSORS	26
3.24 CCR 1 WORKFLOW SCHEDULING RANKING PHASE	27
3.25 CCR 1 WORKFLOW ALLOCATION PHASE ON 3 PROCESSORS	28
3.26 CCR 10 WORKFLOW SCHEDULING AT RANKING PHASE	30
3.27 CCR 10 WORKFLOW ALLOCATION PHASE ON 3 PROCESSORS	30
3.28 CCR 1 WORKFLOW SCHEDULING RANKING PHASE	31
3.29 CCR 1 WORKFLOW ALLOCATION PHASE ON 3 PROCESSORS	31
3.30 CCR 1 WORKFLOW SCHEDULING RANKING PHASE	33
3.31 HEFT ALLOCATION PHASE	33
3.32 CCR 1 WORKFLOW SCHEDULING RANKING PHASE	34
3.33 PEFT ALLOCATION PHASE	34
3.34 CCR 0.1 WORKFLOW SCHEDULING RANKING PHASE	36

3.35 CCR 0.1 WORKFLOW ALLOCATION PHASE ON 10 PROCESSORS	36
3.36 CCR 1 WORKFLOW SCHEDULING RANKING PHASE	37
3.37 CCR 0.1 WORKFLOW ALLOCATION PHASE ON 10 PROCESSORS	37
4.1 AVERAGE MAKESPAN FOR DIFFERENT PROCESSORS.....	39
4.2 BEST RATIO FOR DIFFERENT PROCESSORS	39
4.3 AVERAGE MAKESPAN FOR CCR VALUE	40
4.4 BEST RATIO FOR DIFFERENT CCR VALUE.....	40
4.5 AVERAGE MAKESPAN WITH 5 PROCESSORS FOR DIFFERENT CCR VALUES	41
4.6 BEST RATIO WITH 5 PROCESSORS FOR DIFFERENT CCR VALUES.....	41
4.7 AVERAGE MAKESPAN WITH 5 PROCESSORS FOR DIFFERENT NUMBERS OF BRANCHES	42
4.8 BEST RATIO WITH 5 PROCESSORS FOR DIFFERENT NUMBERS OF BRANCHES.....	42
4.9 VISUALIZED LIGO WORKFLOW.....	43
4.10 AVERAGE MAKESPAN FOR LIGO WORKFLOW ON 10 PROCESSORS.....	44
4.11 VISUALIZED MONTAGE WORKFLOW	45
4.12 AVERAGE MAKESPAN FOR MONTAGE WORKFLOW ON 10 PROCESSORS	45
4.13 VISUALIZED CYBERSHAKE WORKFLOW	46
4.14 Average makespan for CyberShake workflow on 10 processors	46



List of Tables

TABLE 4.1 BEST RATIO FOR LIGO WORKFLOW ON 10 PROCESSORS	44
TABLE 4.2 BEST RATIO FOR MONTAGE WORKFLOW ON 10 PROCESSORS.....	45
TABLE 4.3 BEST RATIO FOR CYBERSHAKE WORKFLOW ON 10 PROCESSORS	46



Chapter 1. Introduction

Workflow, or sometimes called parallel task graph, has long been an important computational model [27][28][29] in the field of parallel processing and its scheduling problem is known to be a challenging NP-complete problem [26]. In general, common workflows usually can be represented by directed acyclic graphs (DAG) for describing inter-task computational structure, as shown in Figure 1.1. Each node in a workflow represents a task that performs computational work by executing a specific program. Each node is annotated with a value indicating the required execution time for the task. The edges between tasks represent their computational interdependencies and thus enforce the execution order among them. The number next to an edge indicates the required data transfer time between tasks. The workflow scheduling problem on a parallel computing platform is to determine the start time and allocated processor for each task in a workflow for optimizing a specific goal, e.g. execution makespan.

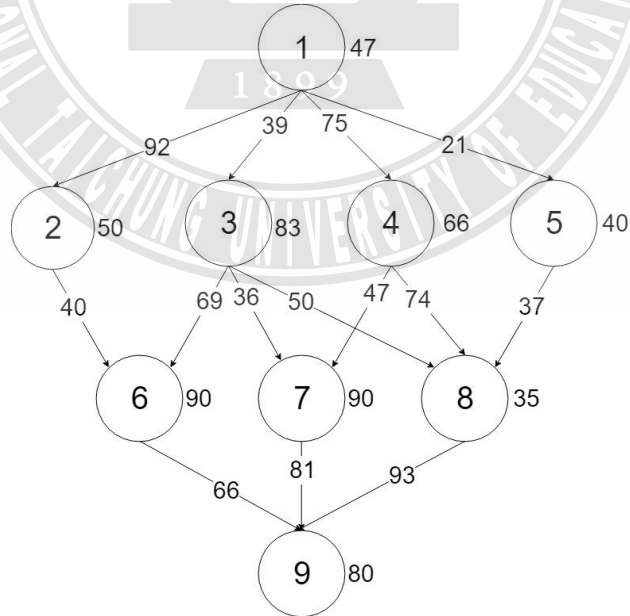


Figure 1.1: workflow structure represented by a DAG

Workflow scheduling problems are often divided into two categories: static scheduling and dynamic scheduling [2]. In the static scheduling problems, all information about task execution time and inter-task communication cost is known and scheduling decisions are made before a workflow starts to run. On the other hand, in dynamic scheduling problems, such information is not available and scheduling decisions are made at runtime. In this thesis, we deal with a static workflow scheduling problem, aiming to minimize the workflow execution makespan.

Static workflow scheduling algorithms can be further divided into two categories, namely heuristic based and guided random search based algorithms [1]. Heuristic-based algorithms [1][2][3] has the advantage of lower computational complexity, while guided random search based algorithms are promising to find better schedules at the cost of longer time for making scheduling decisions. We focus on heuristic based workflow scheduling algorithms in this thesis.

In general, there are three types of heuristic-based workflow scheduling algorithms: clustering-based [18][19], duplication-based [20][21], and list-based [1][2][3] algorithms. Clustering-based workflow scheduling algorithms divide all tasks in a workflow into multiple groups, and try to allocate the tasks belonging to the same group onto the same resource, e.g. processor, aiming to reducing inter-task communication time. Duplication-based algorithms adopt another approach to reducing inter-task communication time, duplicating the execution of a task onto each processor where some other task requires its computational results. Although both clustering-based and duplication-based algorithms have good performance for scheduling a single workflow containing serial tasks, they are not applicable to or have drawbacks in other circumstances. For example, cluster-based algorithms cannot deal with workflows containing parallel tasks and duplication-based algorithms would result in extra computational overheads which could hurt the overall execution performance

especially when scheduling multiple workflows. Therefore, list-based workflow scheduling algorithms become the most widely used approaches in most circumstances. In this thesis, we explore list-based workflow scheduling and propose new scheduling algorithms.

List-based workflow scheduling algorithms consist of two major parts: task ranking and task allocation. Task ranking determines the sequence for allocating all tasks in a workflow onto processors by adopting specific task prioritizing mechanisms. Task allocation selects the most appropriate resource for executing each task in order to benefit the overall workflow execution. Different list-based workflow scheduling algorithms differ in how they conduct task ranking and allocation [1][2][3]. HEFT [1] and PEFT [2] are two most important and well-known list-based workflow scheduling algorithms, and several later algorithms were extended from them [3][6][13]. However, they do not take into consideration workflow structure and parallel system scale when making scheduling decisions. In this thesis, we analyze the strength and weakness of HEFT [1] and PEFT [2] in different circumstances first, and then develop a new workload- and resource-aware list-based workflow scheduling algorithm which is shown superior to HEFT [1] and PEFT [2] in the experiments.

Most previous list-based workflow scheduling algorithms, including HEFT [1] and PEFT [2] were developed for scheduling workflows on heterogeneous parallel computing platforms since heterogeneity is a common characteristic in traditional distributed computing environments. However, in this thesis we focus on the workflow scheduling problem on homogeneous parallel computing platforms since it is now convenient to construct such computing environments on current cloud platforms with virtualization technologies. Moreover, the workflow scheduling problem is NP-complete even for homogeneous parallel computing platforms [26]. The assumption of homogeneity not only conforms to the properties of current cloud computing platforms,

but also allows us to focus on the underlying intrinsic effects of task ranking which are unclear when dealing with heterogeneous computing platforms because of the uncertainty of task execution time on making scheduling decisions.

The remainder of this thesis is organized as follows. Chapter 2 discusses related work in the research field of workflow scheduling. Chapter 3 compares the advantages and disadvantages of HEFT [1] and PEFT [2] under various circumstances, and presents our new workload- and resource-aware list-based workflow scheduling approach. Chapter 4 evaluates the proposed approach with simulation experiments. Chapter 5 summarizes the thesis and discusses some future research directions.



Chapter 2. Related Work

Due to the complex computational structure, workflow computing has been widely applied in modern large-scale scientific and engineering applications. Scheduling becomes an important issue and research topic as it affects the performance and other QoS aspects of workflow execution greatly. A common goal of workflow scheduling is to shorten the time interval between the start of the entry task and the end of the exit task, also called makespan, as much as possible.

Workflow scheduling is a challenging NP-complete problem [24][25][26], and many scheduling algorithms and heuristics have been proposed to tackle it. Among them, list-based workflow scheduling [1][2] is the most widely used category and have been applied to workflows of different structures or computational behaviors for achieving various optimization goals.

Regarding workflow structure, in general it can be divided into three types: task-parallel workflows, mixed-parallel workflows of rigid tasks, and mixed-parallel workflows of moldable tasks. HEFT [1] and PEFT [2] are well-known list-based scheduling algorithms for task-parallel workflows where each task is conducted by executing a serial program on a single processor. For mixed-parallel workflows, each task could be a data-parallel program requiring more than one processor for execution. However, some mixed-parallel workflows contain rigid tasks only, whose processor requirements are fixed and specified by users submitting them. On the other hand, some mixed-parallel workflows might consist of moldable tasks which are flexible in the required numbers of processors for execution and allow the decisions to be made by the workflow scheduler. List-based algorithms have been applied to schedule mixed-parallel workflows containing rigid tasks in [31][32] and mixed-parallel workflows with moldable tasks in [33][34].

Regarding computational behavior, some algorithms were developed for scheduling a single workflow [1][2], while there are also scheduling methods aiming to deal with multiple workflows all ready at the same time [35][36]. A specific category of scheduling algorithms was designed to schedule a set of workflows in an online manner where workflows arrive at different time instants [37][38]. There are various optimization goals for different workflow scheduling algorithms. The most common goal is to minimize the workflow execution makespan as in [1][2]. As cloud and services computing platforms emerge, energy consumption [39][40] and monetary cost [41][42] are also becoming important optimization goals.

In this thesis, we focus on list-based scheduling of a single task-parallel workflow since it involves the intrinsic issues of the workflow scheduling problem and still has potential for further performance improvement. Heterogeneous Earliest finish Time (HEFT) [1], its lookahead variant [3], and Predicted Earliest Finish Time (PEFT) [2] are three important previous works in this field. The goal of our research work is to develop new workflow scheduling algorithms outperforming the three well-known methods.

2.1 Heterogeneous Earliest Finish Time (HEFT)

HEFT [1] is one of the most well-known workflow scheduling algorithms because of its low time complexity and good enough schedule quality. The HEFT algorithm has two phases: task ranking and task allocation. In the first phase, each task is ranked according to the following formula (2.1). The rank of each task represents the length of the longest path from the task to the exit task, including the computational cost of the task itself.

$$rank_u(t_i) = \overline{w_i} + \max_{j \in succ(n_i)} (\overline{c_{i,j}} + rank_u(t_j)) \quad (2.1)$$

In formula (2.1), w_i is the computational cost of task t_i , $succ(t_i)$ represents the

successors of task t_i , $c_{i,j}$ is the communication cost between task t_i and task t_j . The rank value of the exit task in a workflow is simply its computation cost. The result of the task ranking phase is a list of all tasks in a workflow sorted by their rank values. The list is then used to determine the processor allocation sequence in the later task allocation phase. In the task allocation phase, each task's start time and allocated processor is determined. Each task would be assigned to the processor which can finish its execution earliest. HEFT uses an insertion strategy to attempt to schedule a task onto the earliest idle time slot on the allocated processor.

2.2 A Lookahead Variant of HEFT

A Lookahead variant of HEFT was proposed in [3], which could produce better workflow schedules than HEFT, but incurs higher computational complexity because it needs to evaluate the impact on all child tasks before making a task's allocation decision. In [3], four different possibilities are proposed to enhance HEFT through forward-looking information.

1. Lookahead. The resource selected for a task is the resource that minimizes the maximum EFT of all its children, where its children are allocated according to the EFT principle.

2. Lookahead with weighted average EFT. The resource selected for a task is the resource that minimizes the weighted average EFT of all its children. The weight for each task is its rank value.

3. Lookahead and priority list change. If the first two unscheduled ready tasks (ranked by HEFT) are independent, then an evaluation of scheduling these two tasks independently would be conducted to determine actually which one should be scheduled first based on the maximum EFT of their children. The task with smaller children's EFT would be scheduled before the other task.

4. Lookahead and priority list change with weighted average EFT. This method adopts the same priority list change mechanism as the previous method, but uses weighted average EFT instead of maximum EFT.

2.3 Predict Earliest Finish Time (PEFT)

PEFT [2] is an algorithm claimed to maintain the same time complexity as HEFT [1] and achieve comparable or even better performance than the lookahead variant of HEFT [3]. PEFT gains lookahead information by establishing an Optimistic Cost Table (OCT) prior to task ranking. OCT is a two-dimensional table indexed by a pair of task and processor. Each entry in OCT, e.g. $OCT(t_i, p_k)$, records the maximum value of the shortest paths from all the children of task t_i to the exit task once task t_i is allocated onto processor p_k . For each processor, the exit task's OCT value is set to zero. Starting from the exit task, each entry of OCT can be calculated recursively according to the following formula.

$$OCT(t_i, p_k) = \max_{t_j \in succ(t_i)} (\min_{p_w \in P} (OCT(t_j, p_w) + w(t_j, p_w) + \overline{c_{i,j}}))$$

where $\overline{c_{i,j}} = 0$ if $p_w = p_k$ (2.2)

In the task ranking step, PEFT uses OCT to determine the rank of each task as defined in formula (2.3).

$$rank_{oct}(t_i) = \frac{\sum_{k=1}^P OCT(t_i, p_k)}{P} \quad (2.3)$$

In the task allocation step, an Optimistic EFT (O_{EFT}) value for each task on each processor is calculated as the sum of the earliest completion time of the task on the processor according to formula (2.4).

$$O_{EFT}(t_i, p_k) = EFT(t_i, p_k) + OCT(t_i, p_k) \quad (2.4)$$

Based on the calculated O_{EFT} , the task would be assigned to the processor with the minimum O_{EFT} value. The EFT plus OCT in PEFT may not select the processor that leads to the earliest completion time for the current task, but is promising to shorten the makespan of the entire workflow.

Chapter 3. List-Based Workflow Scheduling

This chapter first presents detailed analysis of the two most important list-based workflow scheduling algorithms, i.e. HEFT [1] and PEFT [2], based on a series of simulation environments configured with different workflow structures, workload levels, and numbers of resources. Relative strength and weakness of HEFT and PEFT under different circumstances are discussed. Based on the analysis and discussions, a new workload- and resource-aware list-based workflow scheduling algorithm is then proposed to overcome the weakness of previous methods and is expected to achieve better workflow execution performance.

3.1 Comparative Analysis of HEFT and PEFT

In the following experiments, workflows of typical and common fork-join structure [17] are used to evaluate and compare HEFT [1] and PEFT [2]. Each workflow contains a single entry node and a single exit node. The number of branches in each fork-join structure is randomly generated between one to three, and there might be at most two layers of fork-join structure within a workflow. Therefore, the maximum degree of parallelism in workflow is nine. Experiments with different numbers of processors and different workflow properties, e.g. Communication-to-Computation Ratio (CCR) [26], were conducted to compare HEFT [1] and PEFT [2] under various workload and resource conditions. In each experiment, different algorithms were used to schedule the same 1000 workflows and then their corresponding performance results were compared and analyzed.

Two performance metrics, average makespan of 1000 jobs and best ratio, were used for comparing HEFT [1] and PEFT [2]. Makespan is a widely used performance

metric in most workflow scheduling research papers, which, measure the time period from the start of the entry task to the finish of the exit task. A shorter workflow execution makespan is usually used to indicate the superiority of a specific scheduling algorithm. Average makespan of all jobs is a useful system-wide performance metric most system administrators might be interested in. On the other hand, users of individual workflows might be more interested in the best ratio since it tries to measure the probability that a specific scheduling algorithm would produce the best schedule for any workflow by counting the number of workflows that have the shortest makespan in the experiment and then dividing it by 1000.

Figures 3.1 and 3.2 present the experimental results for three, five, and ten processors in terms of average makespan and best ratio, respectively. In each figure, performance results across workflows of different CCR values are compared.

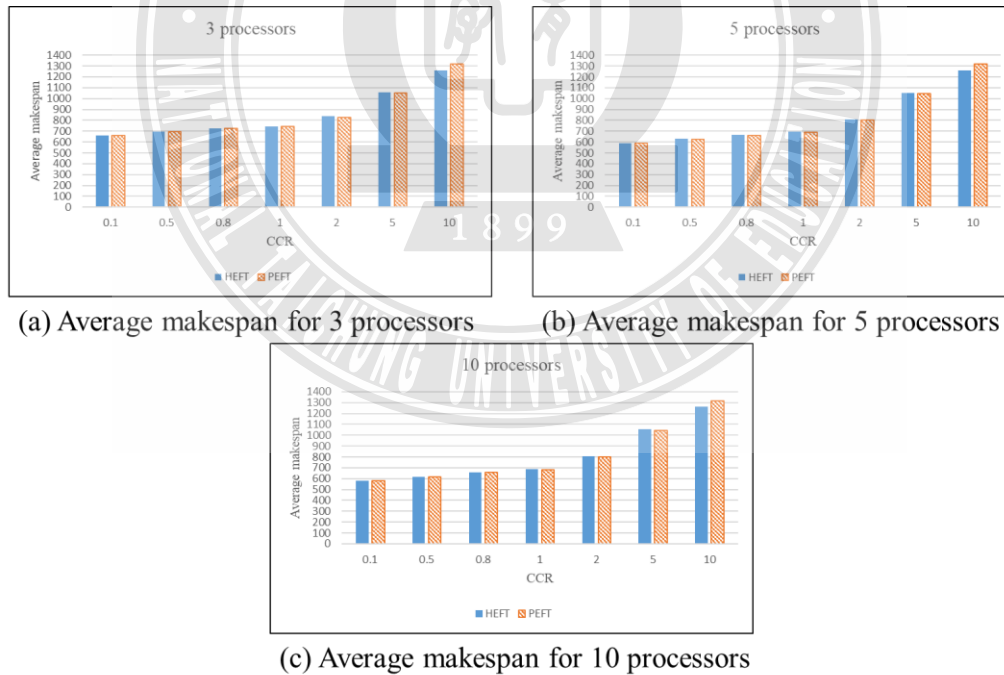


Figure 3.1: Average makespan for different processors

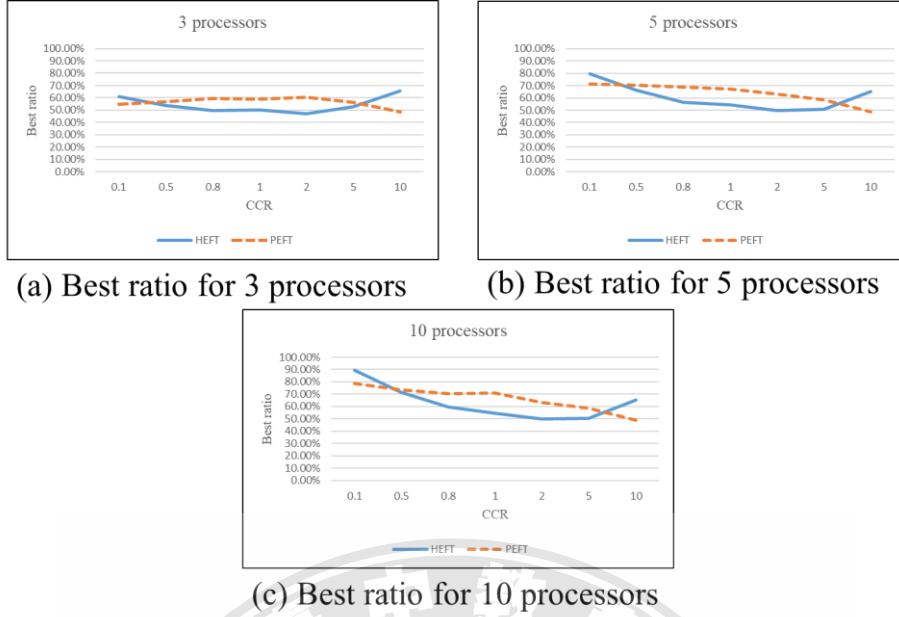


Figure 3.2 Best ratio for different processors

In [2] experimental results were presented to demonstrate that PEFT is superior to HEFT. However, in our detailed analysis Figures 3.1 and 3.2 indicate that PEFT [2] only has advantage over HEFT [1] for workflows with medium CCR values, but results in worse performance than HEFT [1] for lower or higher CCR values. Figures 3.3 and 3.4 present performance comparison in another perspective. Each figure shows the experimental results of workflows with a specific CCR value running across different numbers of processors. The performance comparison in Figures 3.3 and 3.4 clearly confirms the trend found in Figures 3.1 and 3.2. The relative strength and weakness of HEFT [1] and PEFT [2] change across different CCR values.

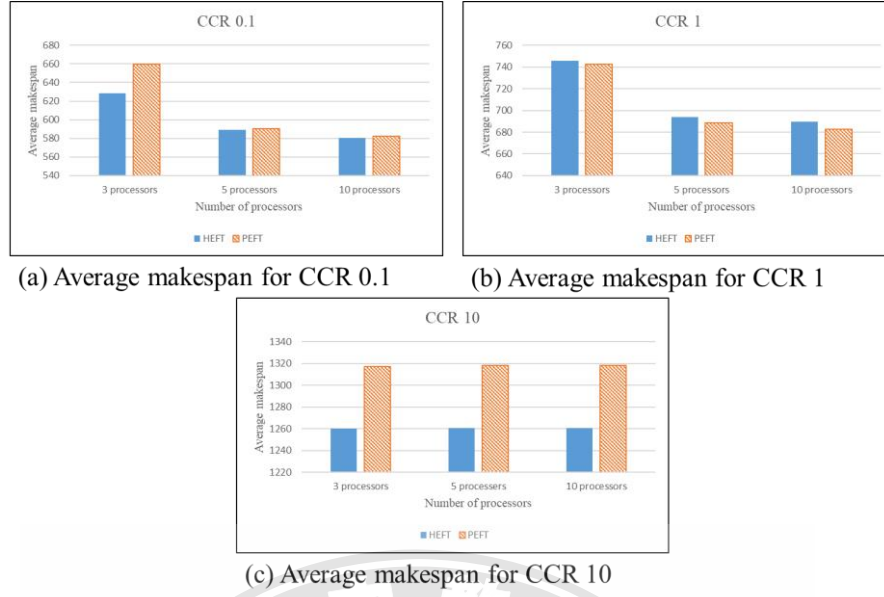


Figure 3.3: Average makespan for different CCR values

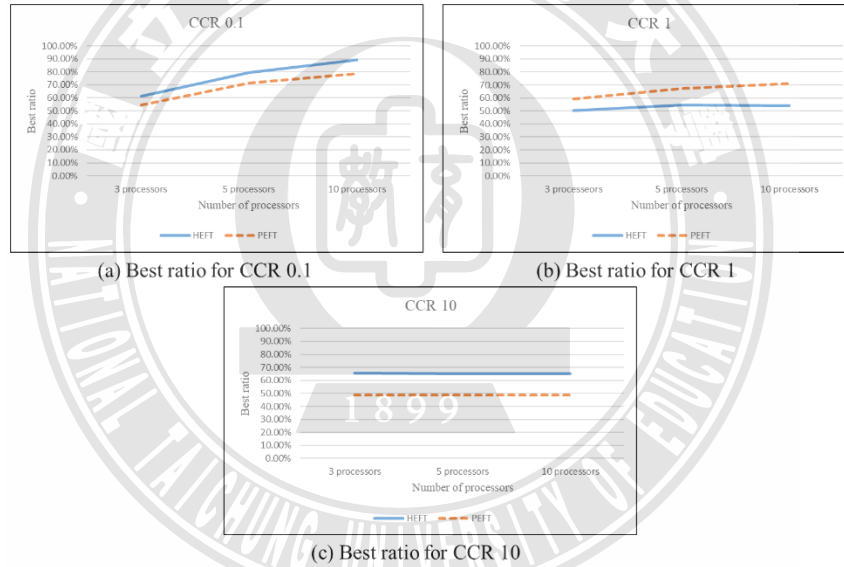


Figure 3.4: Best ratio for different CCR values

Figures 3.5 and 3.6 evaluate the impact of workload levels on different scheduling algorithms. In the figures, the number of branches in a workflow indicates the maximum degree of parallelism within it. If the degree of parallelism is less than the number of available processors, the workload level is regarded as low. On the other hand, if the number of processors is smaller than the degree of parallelism, the workload level would be high. The experimental results shown in Figures 3.5 and 3.6 indicate that workload levels have little impact on the relative performance between HEFT [1]

and PEFT [2]. Across three different workload levels, PEFT [2] always has performance advantage over HEFT [1] only for workflows of medium CCR values. Figures 3.7 and 3.8 present performance analysis in another perspective, and confirm the trend found in Figures 3.5 and 3.6. It is obvious from Figure 3.8 that PEFT [2] is superior to HEFT [1] for medium CCR values. As shown in Figures 3.6 and 3.8, one important thing to be noted is the probability that HEFT [1] and PEFT [2] produce schedules of the same makespan would decline as the workload level increases, i.e. a larger number of branches than available processors.

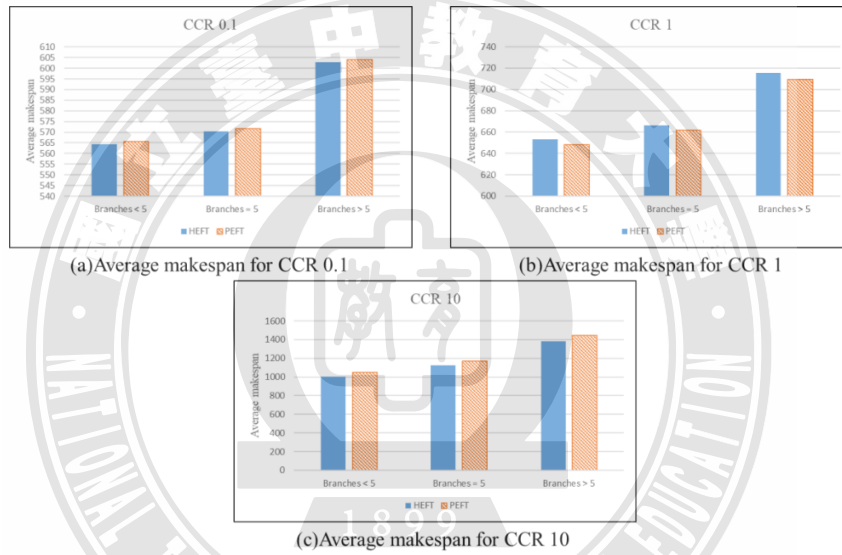


Figure 3.5: Average makespan with 5 processors for different CCR value

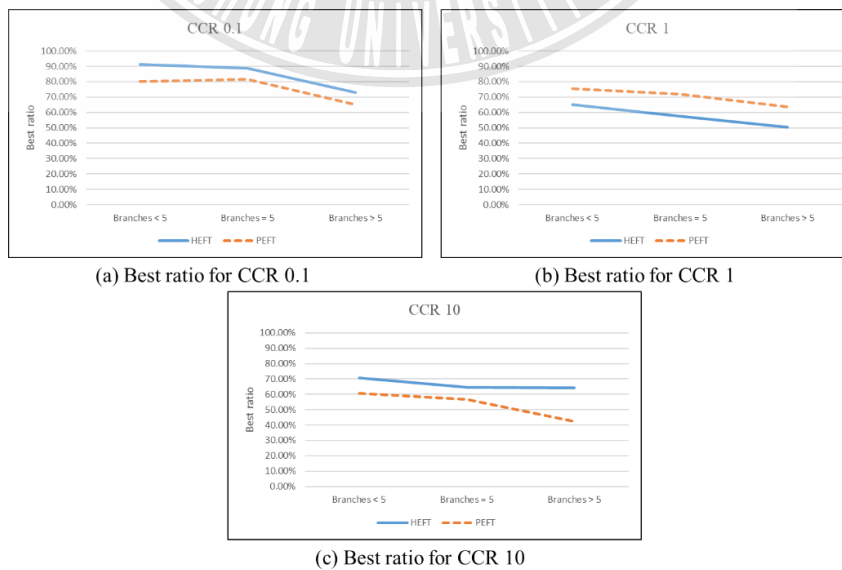


Figure 3.6: Best ratio with 5 processors for different CCR value

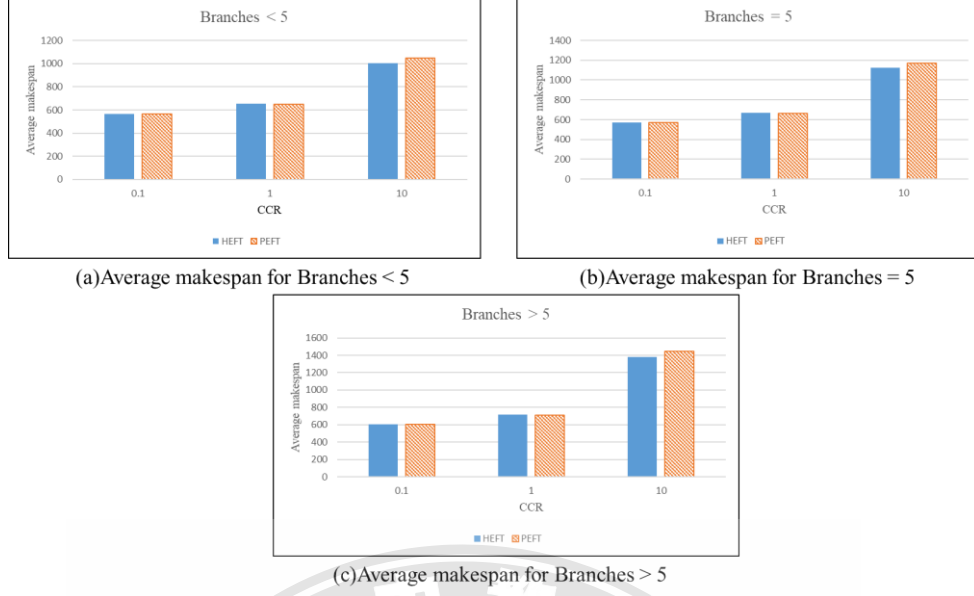


Figure 3.7: Average makespan with 5 processors for different Branches

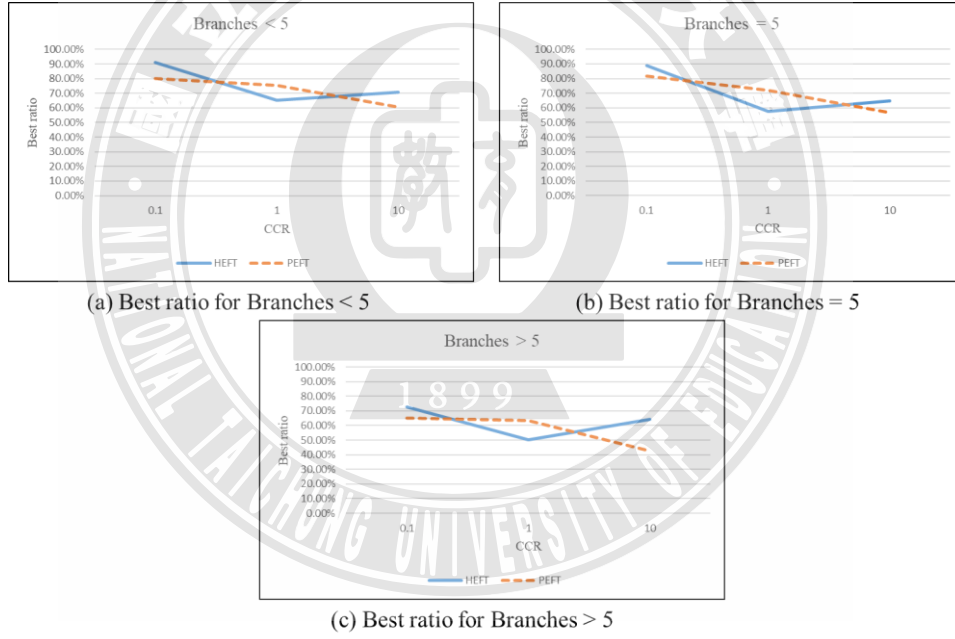


Figure 3.8: Best ratio with 5 processors for different Branches

In general, the potential advantage of PEFT [2] over HEFT [1] comes from two parts. The first is that it tries to calculate a more accurate bottom rank [26] value based on its optimistic cost table which estimates the potential elimination of inter-task communication cost, while HEFT [1] considers all inter-task communication costs. The second is involved in the task allocation phase where PEFT [2] takes into consideration the resultant performance of each task's successors, while HEFT [1] doesn't. However,

since the potential elimination of inter-task communication is just an estimation in PEFT [2], it would sometimes result in better schedules than HEFT [1] when the estimation is close to the later actual schedule, but run into worse schedules when the estimation is not accurate enough.

In the following, we present several workflow scheduling examples, selected from the above experiments, to analyze and illustrate the relative strength and weakness between HEFT [1] and PEFT [2].

3.1.1 CCR equal to 0.1

Figures 3.9 and 3.10 present an example on three processors where the inaccurate estimation in the optimistic cost table of PEFT [2] leads to a worse schedule than HEFT [1]. Figure 3.9 shows the information about workflow structure, computation and communication costs, and the task ranking results by HEFT [1] and PEFT [2], respectively. Figure 3.10 presents the resultant schedules produced by HEFT [1] and PEFT [2], respectively.

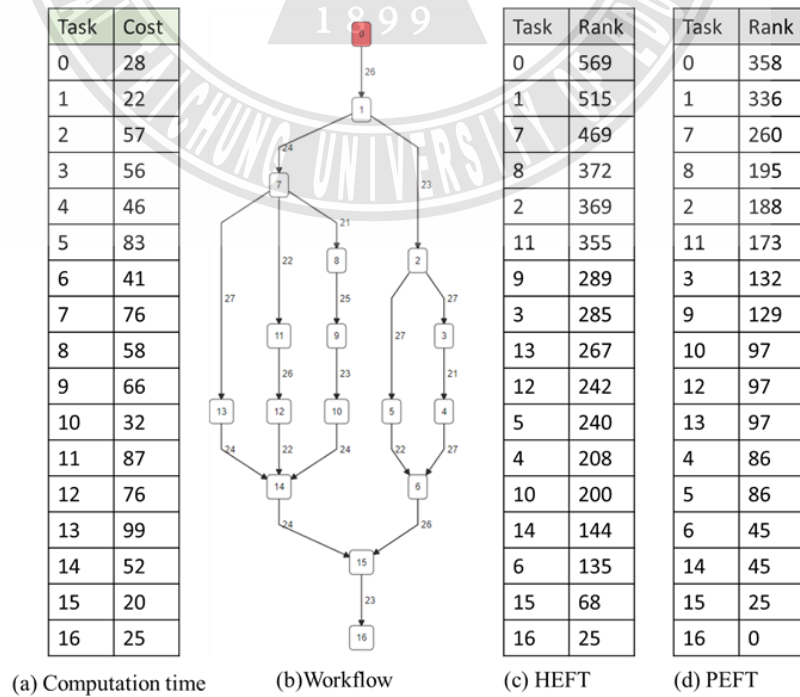


Figure 3.9: CCR 0.1 workflow scheduling ranking phase

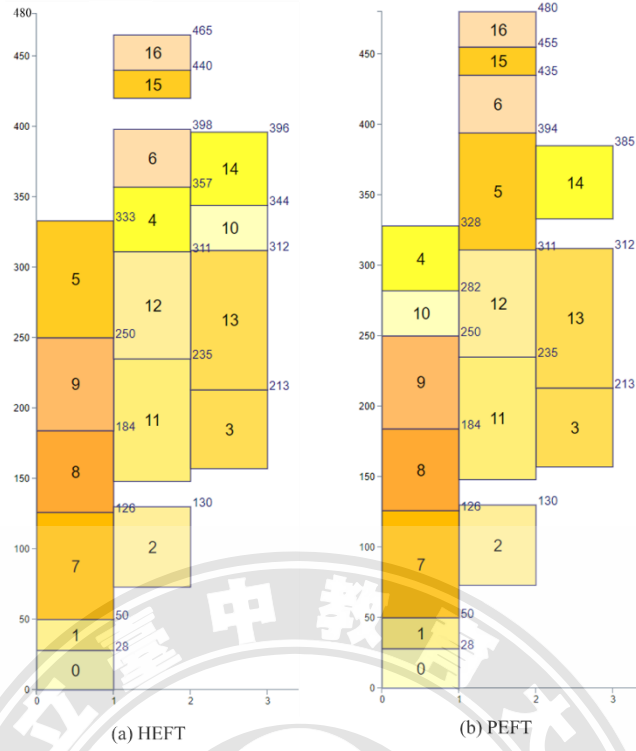


Figure 3.10: CCR 0.1 workflow allocation phase on 3 processors

The critical tasks that lead to the resultant different schedules are tasks 5 and 10. In HEFT, task 5 is given a higher rank value, while in PEFT the rank value of task 10 is higher than task 5 because more communication costs along the paths from task 5 to the exit task are estimated to be eliminated. As shown in Figure 3.10, it turns out that allocating task 5 first is a better choice since the summation of the computation costs of all tasks on the path between it and the exit task is significantly larger than that on the path from task 10 to the exit task. Therefore, HEFT produces a better schedule than PEFT in this case.

Figures 3.11 and 3.12 presents another example where the workflow has a larger number of branches and runs on more processors. In this case, the critical tasks resulting in the major difference between the two schedules are tasks 13 and 2. Task 13 has a higher rank value in HEFT, while task 2 does in PEFT. When considering computation costs of tasks only, the path length between task 13 and the exit task is longer than that between task 2 and the exit task. Therefore, allocating task 13 first is a right choice in

this case, but PEFT allocates task 2 first because of its inaccurate estimation of communication cost elimination, resulting in a worse schedule than HEFT.

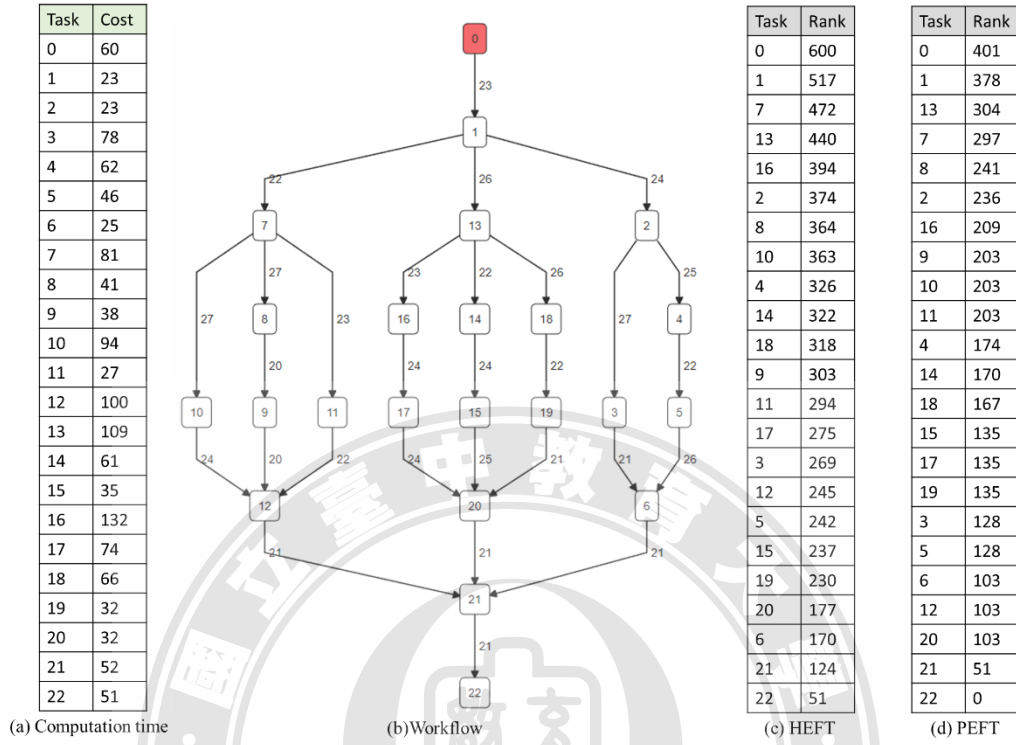


Figure 3.11: CCR 0.1 workflow scheduling ranking phase

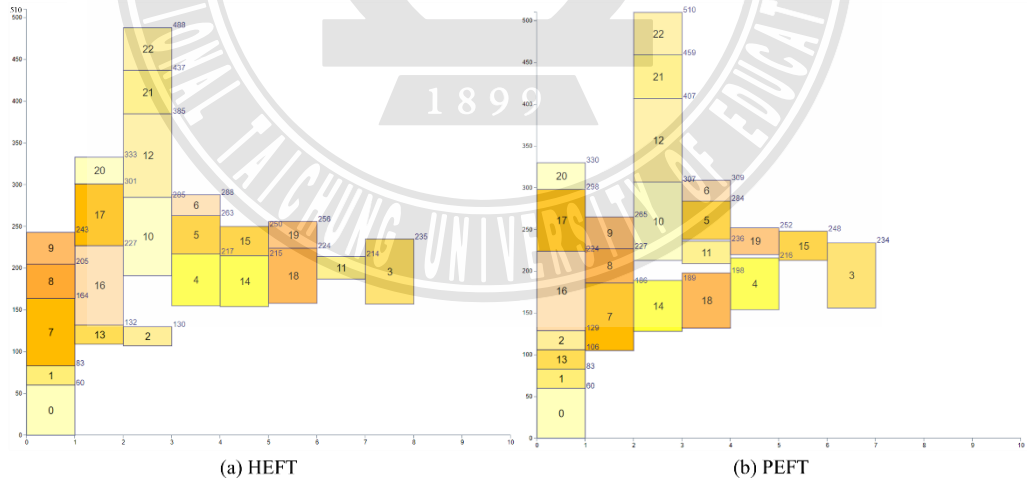


Figure 3.12: CCR 0.1 workflow allocation phase on 10 processors

3.1.2 CCR equal to 1

In the following example shown in Figures 3.13 and 3.14, PEFT produces a better schedule than HEFT. In this case, the critical tasks are tasks 4, 15, and 21 which affect

the final schedule greatly. In contrast to the two examples in the previous section, the three critical tasks have almost equivalent computational bottom rank [26], i.e. the bottom rank considering only tasks' computational costs. Therefore, the estimation of inter-task communication cost elimination in PEFT plays an important role in ranking the three critical tasks correctly, and thus results in a better schedule than HEFT does.

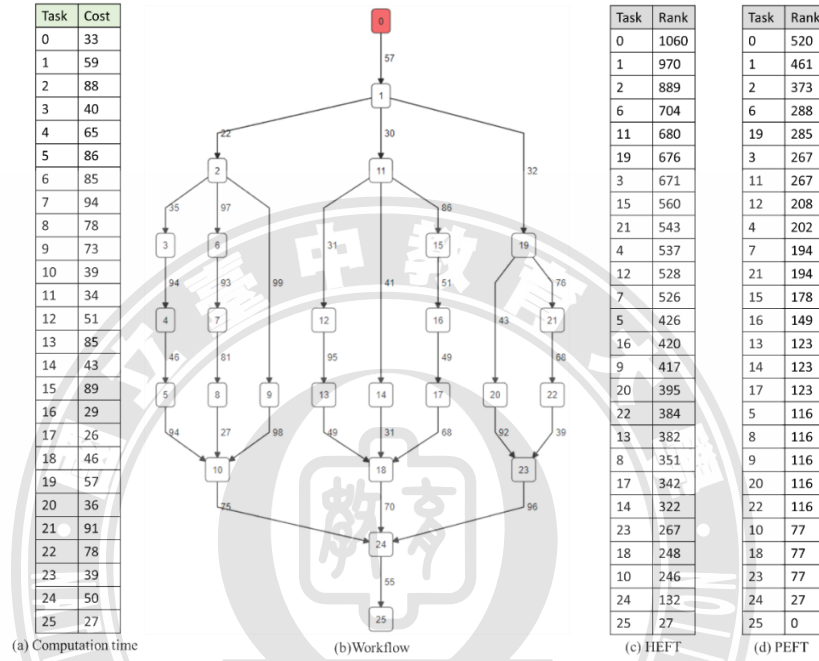


Figure 3.13: CCR 1 workflow scheduling ranking phase

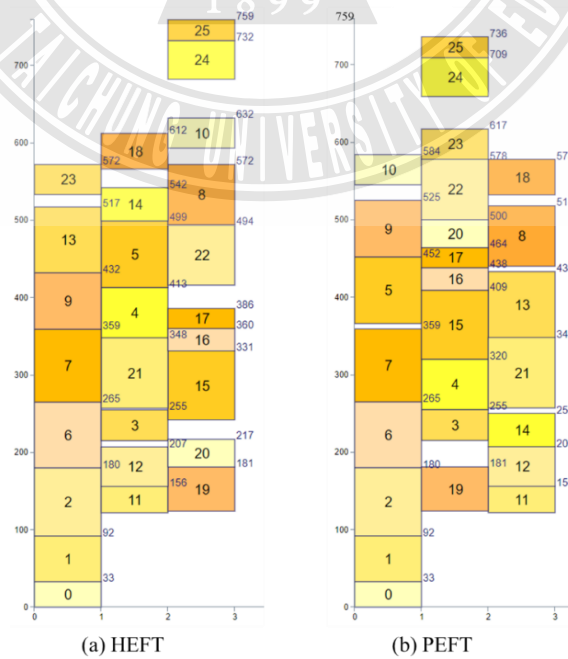


Figure 3.14: CCR 1 workflow allocation phase on 3 processors

Figures 3.15 and 3.16 presents another example with a workflow of fewer branches running on more processors. In this case, tasks 2 and 10 are the critical tasks where the computational bottom rank of task 2 is higher than task 10, but the bottom rank of task 10 is higher than task 2. Therefore, in HEFT task 10 is allocated before task2. On the other hand, PEFT makes a better decision of allocating task 2 first with the help of the estimation of inter-task communication elimination.

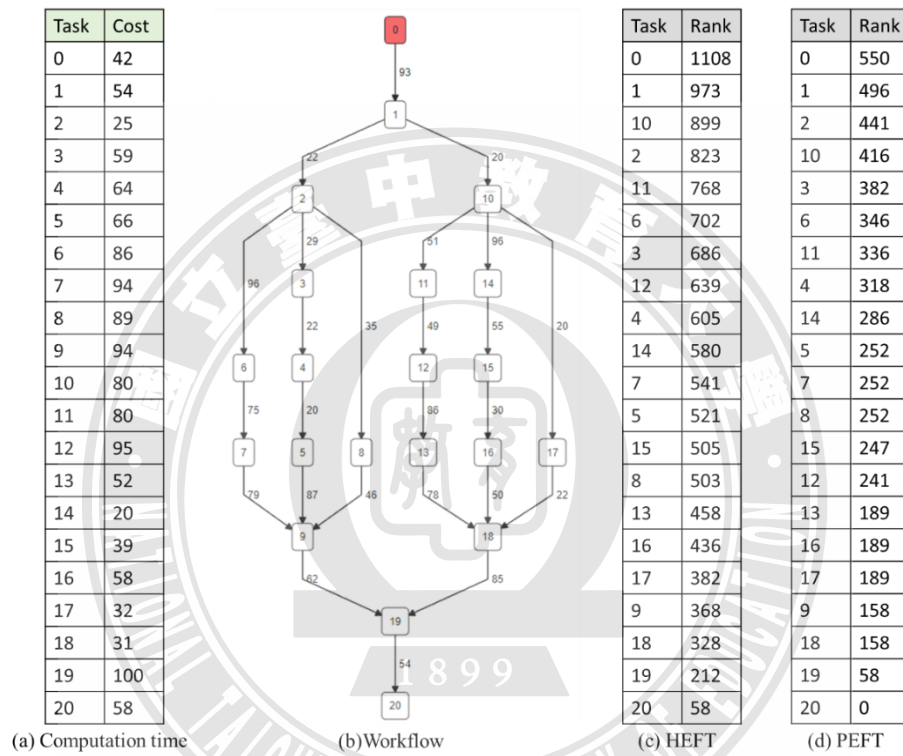


Figure 3.15: CCR 1 workflow scheduling ranking phase

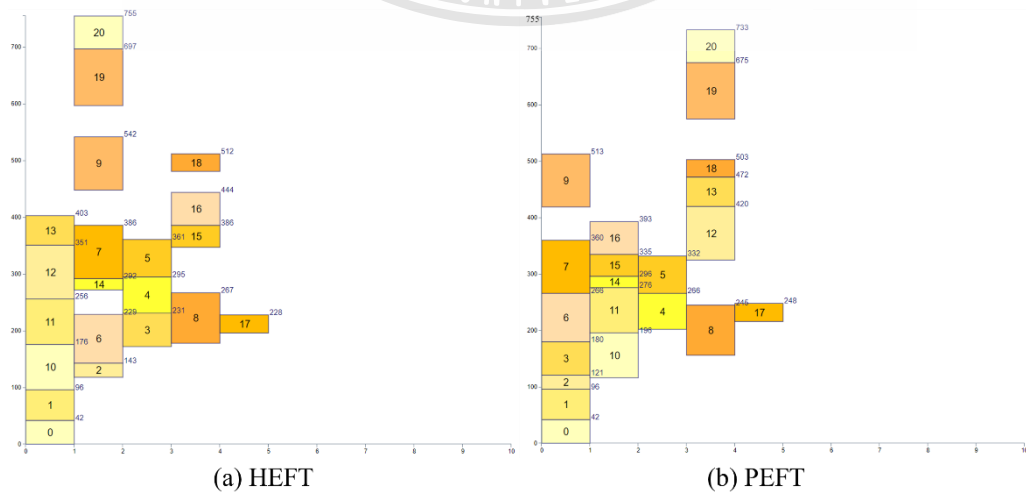


Figure 3.16: CCR 1 workflow allocation phase on 10 processors

3.1.3 CCR equal to 10

As CCR values increase, the estimation of inter-task communication in PEFT plays an even important role on the quality of final schedules, which has potential to lead to better schedules. However, the price is also heavy once the estimation accuracy is low. Figures 3.17 and 3.18 shows such an example. Inaccurate estimation of inter-task communication results in allocating task 14 before task 5 in PEFT, which finally produces a worse schedule than HEFT as shown in Figure 3.18.

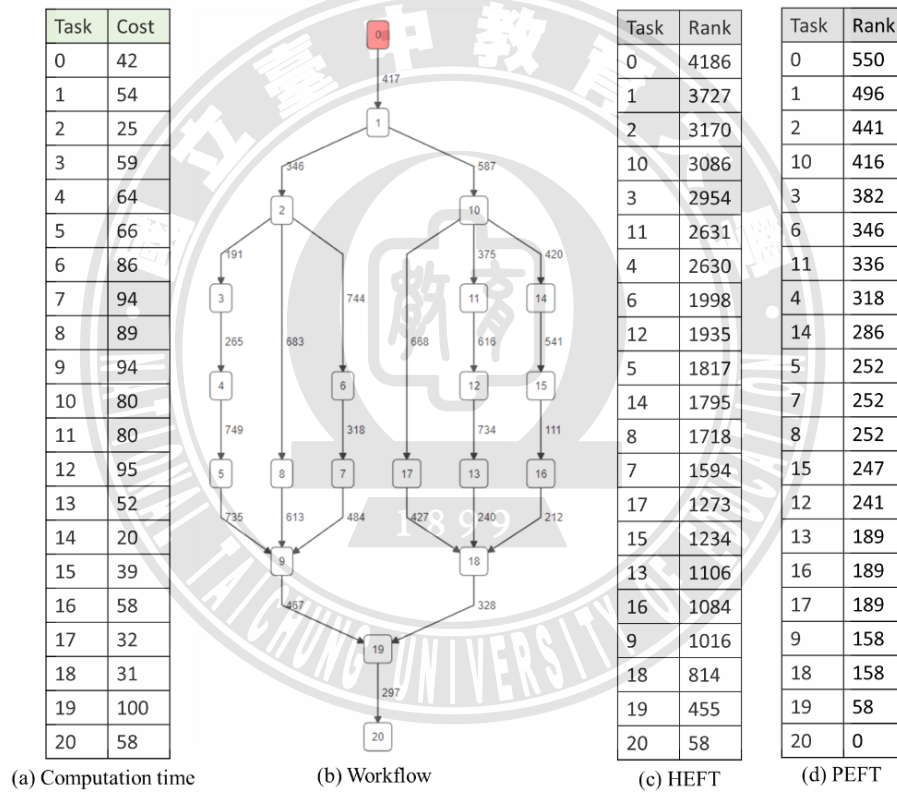


Figure 3.17: CCR 10 workflow scheduling ranking phase

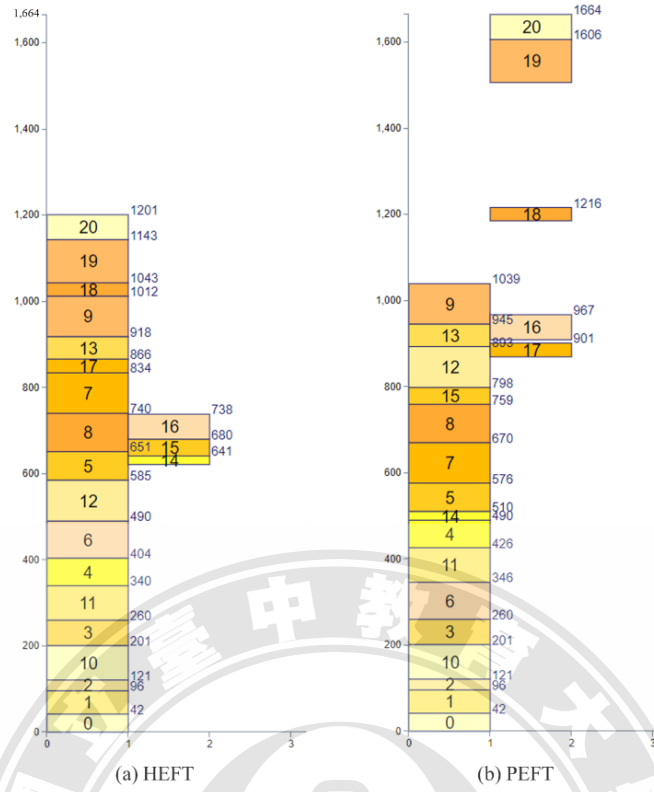


Figure 3.18: CCR 10 workflow allocation phase on 3 processors

Figures 3.19 and 3.20 presents another similar example involving more processors. However, since inter-task communication costs are high for workflows of large CCR values, only a small portion of processors are actually used as shown in Figure 3.20. Therefore, the result is similar to that in the previous example. In this case, tasks 2 and 11 are the critical tasks affecting the final schedules. Inaccurate estimation of inter-task communication in PEFT leads to the earlier allocation of task 11 than task 2. Such arrangement results in the occurrence of a very high communication cost between tasks 2 and 1. As a result, PEFT produces a worse schedule than HEFT in this case.

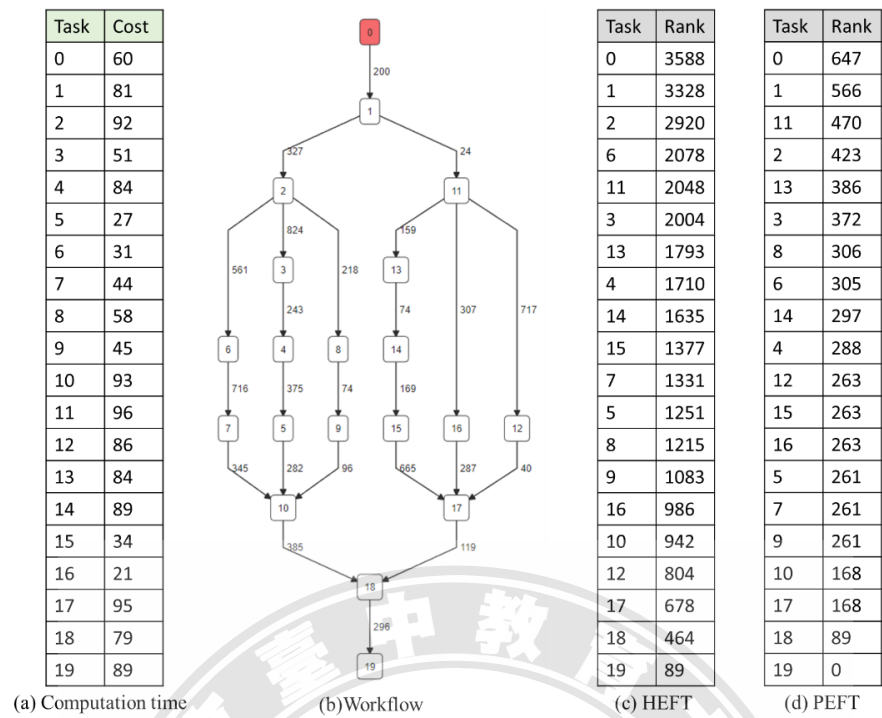


Figure 3.19: CCR 10 workflow scheduling ranking phase

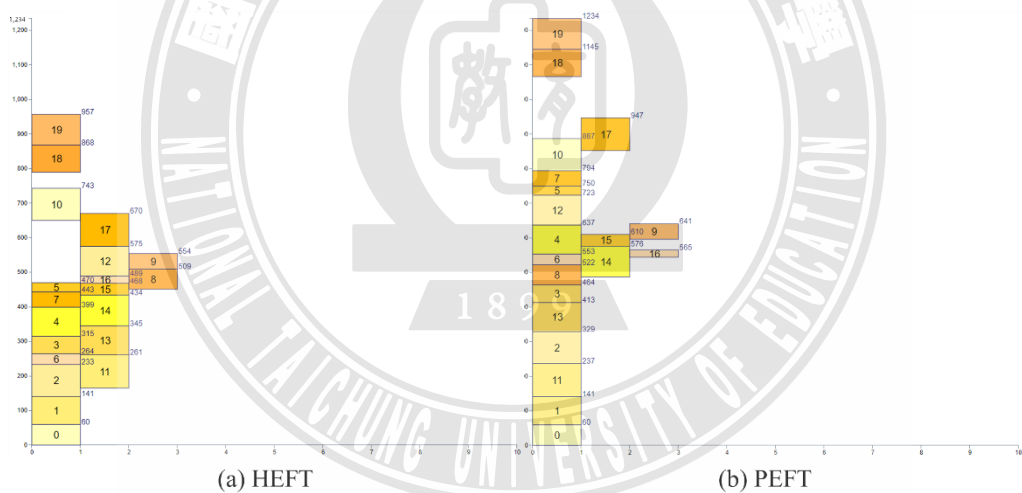


Figure 3.20: CCR 10 workflow allocation phase on 10 processors

3.2 Workload- and Resource-Aware List Scheduling

Approach

Two observations on the experimental analysis presented in the previous section motivated the development of our new workload- and resource-aware list-based workflow scheduling approach.

- 1 It is a common assumption in most workflow scheduling researches that the communication cost between two tasks would reduce to zero once they are scheduled onto the same processor for execution. Since the rank value of each task contains inter-task communication costs along a path starting from it to the exit task, accurate estimation of the inter-task communication costs that would actually occur in the final schedule is crucial. Accurate task rank values could lead to a better task allocation sequence and thus a shorter makespan in the final schedule.
- 2 On each task allocation, it is not good enough to consider only the task itself, e.g. HEFT [1], or its successors, e.g. PEFT [2]. It is beneficial to also take into consideration the effects of a task allocation on other non-successor tasks.

Our new workload- and resource-aware list-based workflow scheduling approach features three new task ranking or allocation mechanisms which are described and discussed in the following sections.

3.2.1 Structural Task Ranking

Most previous list-based workflow scheduling algorithms, including HEFT [1] and PEFT [2], adopts path-oriented task rank calculation. In this section, we present a workload- and resource-aware structural task ranking mechanism, which is expected to more accurately estimate the actual occurrences of inter-task communication costs, and thus find a better task allocation sequence. We identified several important inter-task

relationships in common workflow structure as shown in Figure 3.21.

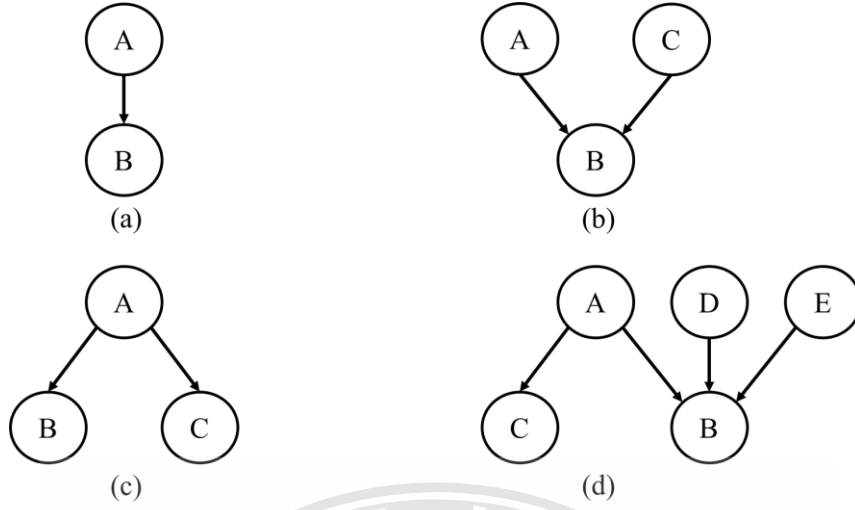


Figure 3.21: Common inter-task relationship in workflow structure

In Figure 3.21 (a), both task A and task B have only one single edge, which connects them together. In this case, it is very likely that they would be assigned to the same processor for execution. Therefore, the communication cost between them is ignored in our structural task rank calculation.

Figure 3.21 (b) shows a join relationship among three tasks. Since tasks A and C are independent, it is usual that they are assigned to different processors for realizing execution parallelism. To minimize potential inter-task communication costs, task B would be assigned to either the processor executing task A or the processor running task C. Therefore, the communication costs on edge (A, B) and edge (C, B) usually would not occur simultaneously. To capture such a property, in our structural task rank calculation, the communication cost between tasks A and B is computed according to the following formula.

$$\frac{\min(\text{PreBra}(t_B), P) - 1}{\min(\text{PreBra}(t_B), P)} \times \overline{C_{A,B}} \quad (3.1)$$

In formula (3.1), $\text{PreBra}(t_B)$ represents the number of task B's incoming edges, and P represents the total processors of the system. Since formula (3.1) takes into consideration the relationship between execution parallelism of a workflow and the

number of processors in a system, it is a kind of workload- and resource-aware estimation of inter-task communication costs.

Figure 3.21 (c) is a structure similar to Figure 3.21 (b), but represents a fork inter-task relationship among three tasks, where task A is the fork task. In this structure, the communication cost between tasks A and C is estimated based on formula (3.2), where $ChildBra(t_A)$ is the number of outgoing edges from task A.

$$\frac{\min(ChildBra(t_A), P) - 1}{\min(ChildBra(t_A), P)} \times \overline{C_{A,B}} \quad (3.2)$$

Figure 3.21 (d) shows a more complex structure containing all the three types of inter-task relationship in Figure 3.21 (a), (b), and (c). In this case, when estimating the actual communication cost between tasks A and B, our approach considers both the outgoing edges of task A and the incoming edges of task B as shown in formula (3.3).

$$\left(\frac{\min(ChildBra(t_A), P) - 1}{\min(ChildBra(t_A), P)} \times \frac{1}{\min(ChildBra(t_A), P)} \times \frac{\min(PreBra(t_B), P) - 1}{\min(PreBra(t_B), P)} \right) \times \overline{C_{A,B}} \quad (3.3)$$

Based on the above inter-task communication cost estimation methods, in the task ranking stage of our workflow scheduling approach, formula (3.4) is used to compute the rank value of each task and then determine the task allocation sequence. In formula (3.4), w_i represents the execution time of task t_i , and task t_j is the child of task t_i .

$$rank_{STR}(t_i) = w_i + \max_{t_j \in succ(t_i)} (rank_{spc}(t_j) + STR(t_i, t_j)) \quad (3.4)$$

Figures 3.22 and 3.23 present an illustrative example to show the superiority of our approach over HEFT [1] for a workflow of a small CCR value, i.e. 0.1, since in the previous analysis, HEFT [1] outperforms PEFT [2] for such kind of workflows. As shown in Figure 3.23, our approach could produce a schedule of shorter makespan, i.e. 611, than HEFT [1], i.e. 628. In this example, tasks 13 and 17 are critical. The more accurate task ranking mechanism of our approach allows task 13 to be allocated before task 17, resulting in earlier start time of task 17 and thus a shorter makespan of the entire schedule.

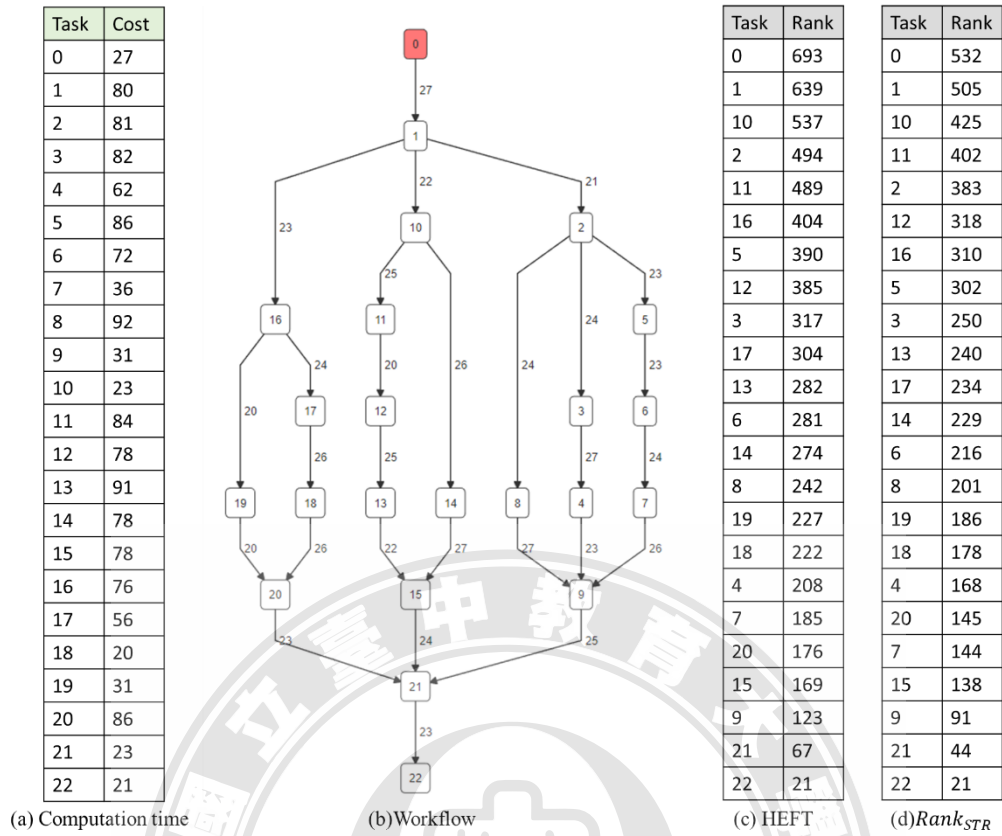


Figure 3.22: CCR 0.1 workflow scheduling ranking phase

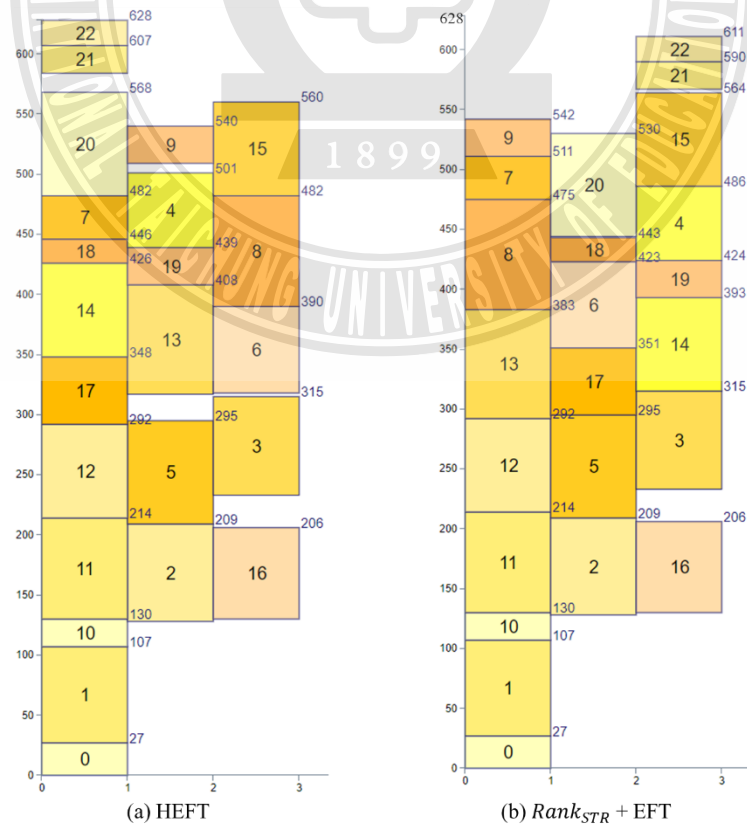


Figure 3.23: CCR 0.1 workflow allocation phase on 3 processors

Figures 3.24 and 3.25 presents another example for workflows of medium CCR values, e.g. 1, where PEFT [2] has advantage over HEFT [1] as shown in previous analysis. Figure 3.25 shows that our approach could produce a schedule of shorter makespan, i.e. 785, than PEFT [2], i.e. 811. In this example, the critical tasks are tasks 6 and 9. The advantage of our task ranking mechanism allows task 9 to be allocated before task 6, leading to earlier start time of task 6 than the start time of task 9 in PEFT's schedule, and thus a shorter makespan than PEFT [2]. The above two examples show that our approach has potential to outperform both HEFT [1] and PEFT [2].

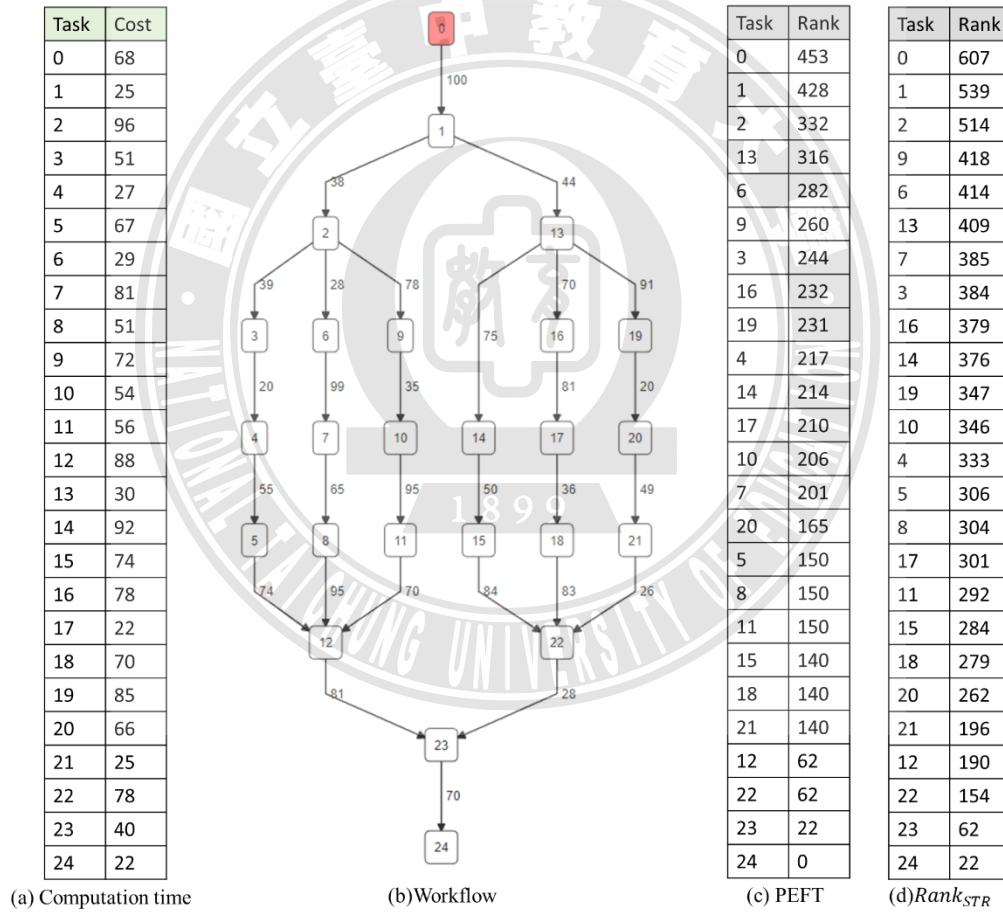


Figure 3.24: CCR 1 workflow scheduling ranking phase

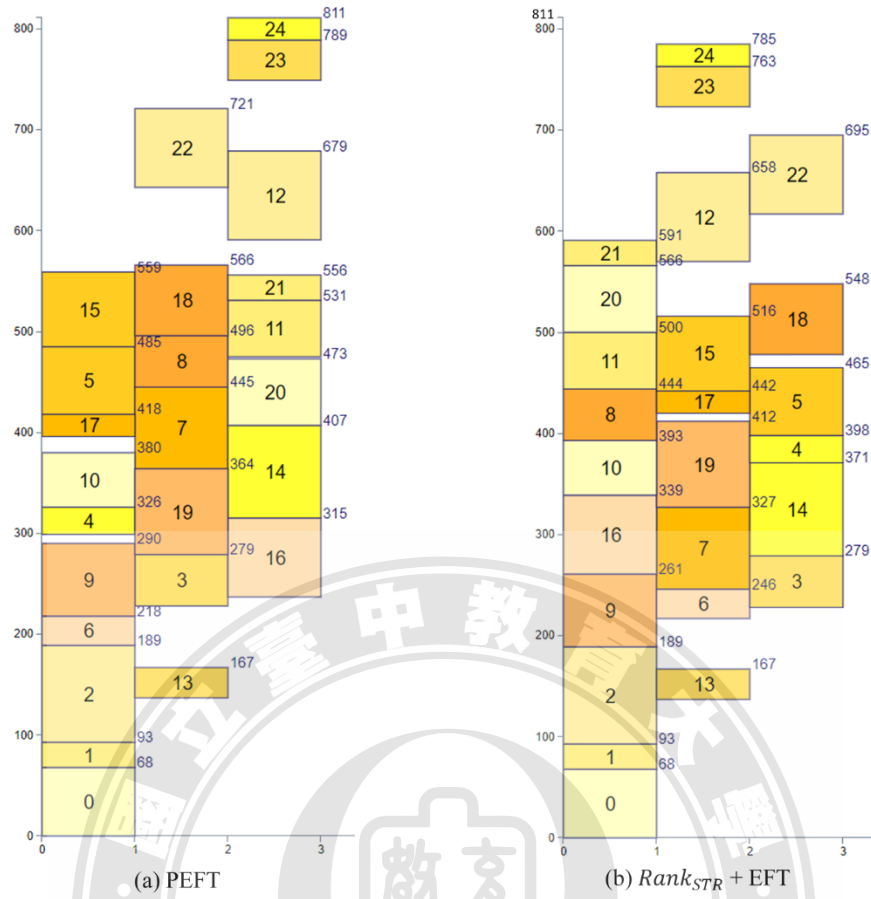


Figure 3.25: CCR 1 workflow allocation phase on 3 processors

3.2.2 Allocated Top Rank

Both HEFT [1] and PEFT [2] adopt the concept of bottom rank [26] in the task ranking stage. They calculate the rank value of each task according the following formula (3.5). The bottom rank of a task represents the longest path from it to the exit task.

$$BR(t_i) = \begin{cases} w_i, & \text{if } t_i \text{ is an exit node} \\ w_i + \max_{t_j \in \text{succ}(t_i)} (\overline{C}_{i,j} + BR(t_j)), & \text{otherwise} \end{cases} \quad (3.5)$$

However, one important technique to minimize the makespan of a workflow execution schedule is reducing the length of critical paths. Therefore, giving higher rank values to tasks on the critical paths would be a promising task ranking method. In order to do that, both top rank [26] and bottom rank [26] should be considered when ranking

tasks since the sum of a task's top and bottom rank values represents the length of the longest paths going through it. The top rank of a task is defined as formula (3.6), which represents the longest path from the entry task to the task [26].

$$TR(t_i) = \begin{cases} 0, & \text{if } t_i \text{ is an entry node} \\ \max_{t_j \in pre(t_i)} (\overline{C_{i,j}} + TR(t_j)), & \text{otherwise} \end{cases} \quad (3.6)$$

One important difference between top and bottom ranks is that when a task becomes ready for execution, the tasks contributing to its top rank have all been allocated, while those tasks resulting in its bottom rank haven't. Since the tasks related to a top rank have been allocated, the inter-task communication costs among them can be computed accurately with the knowledge of which tasks being allocated onto the same processor. Therefore, it is possible to use a more accurate calculation to compute a task's top rank instead of formula (3.6). Such calculation is called allocated top rank and defined as formula (3.7) [26]. In our approach, we compute the rank value of each task by summing up its allocated top rank and bottom rank.

$$ATR(t_i) = \begin{cases} 0, & \text{if } t_i \text{ is an entry node} \\ \max_{t_h \in pre(t_i)} (w_i + \overline{C_{i,h}} + ATR(t_h)), & \text{if } t_i, t_h \text{ on same resource, } \overline{C_{i,h}} = 0 \end{cases} \quad (3.7)$$

Figures 3.26 and 3.27 presents an illustrative example for workflows of large CCR values, e.g. 10, demonstrating the superiority of the new task ranking mechanism over traditional bottom rank used in HEFT [1] and PEFT [2]. In this case, tasks 8 and 14 are critical. With the new task ranking mechanism, task 8 would be allocated before task 14, resulting in earlier start time of task 14 than the start time of task 8 in HEFT schedule and thus a shorter makespan than HEFT as shown in Figure 3.27.

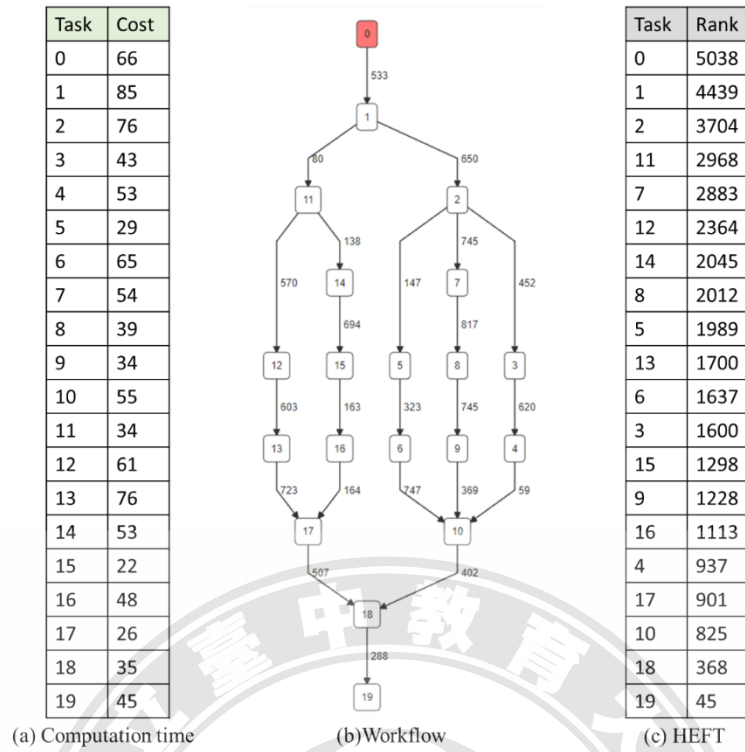


Figure 3.26: CCR 10 workflow scheduling at ranking phase

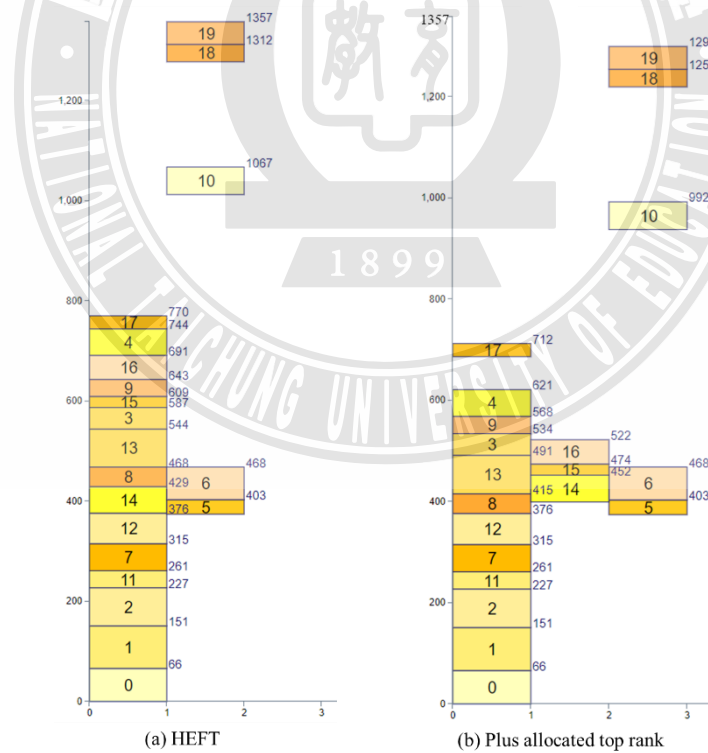


Figure 3.27: CCR 10 workflow allocation phase on 3 processors

Figures 3.28 and 3.29 shows another example for medium-CCR workflows where PEFT [2] has advantage over HEFT [1] in the previous analysis. In this example, the

critical tasks are tasks 3, 8, and 15. With the new task ranking mechanism, our approach would allow task 3 to be allocated before tasks 8 and 15, resulting in earlier start time of task 8 than the start time of task 3 in PEFT and finally a shorter makespan than PEFT [2].

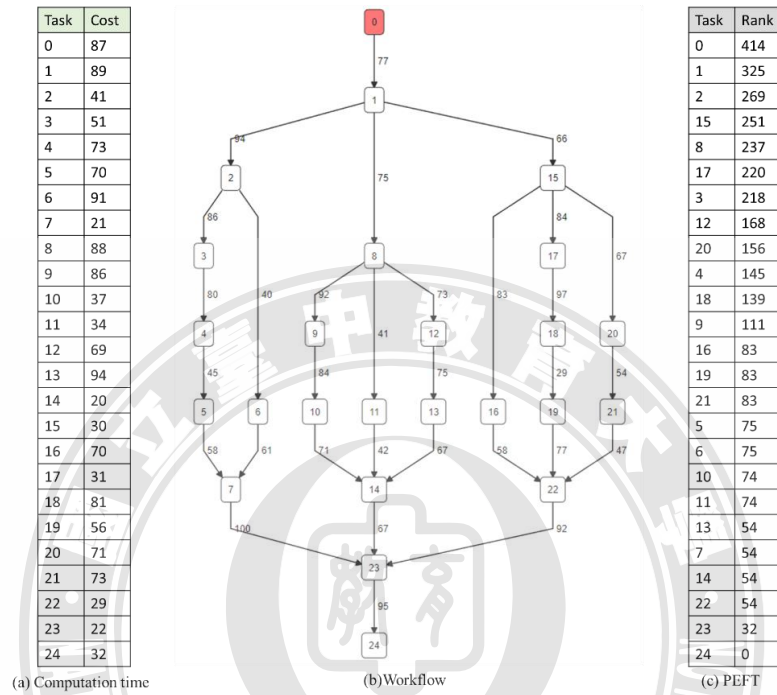


Figure 3.28: CCR 1 workflow scheduling ranking phase

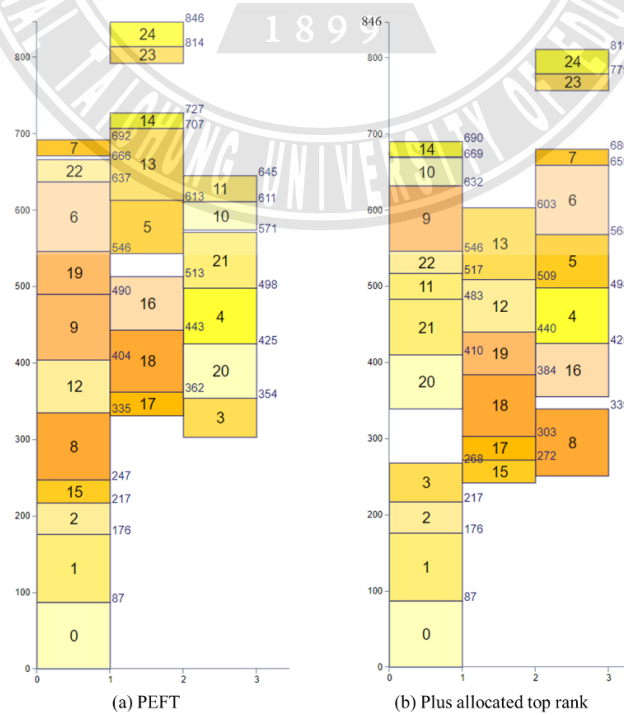


Figure 3.29: CCR 1 workflow allocation phase on 3 processors

3.2.3 Adaptable Task Allocation

When making the processor allocation decision for a task, sometimes there would be more than one processor leading to the same finish time for the task. In HEFT [1], these processors are regarded as equivalent and one of them would be picked up randomly for the allocation. In PEFT [2], these processors would be further compared based on potential impact on the finish time of the task's successors. Therefore, PEFT [2] has potential to achieve better performance than HEFT [1] because of the superior task allocation mechanism. In this section, we present an adaptable task allocation method which takes into consideration the potential impact on other non-successor tasks, aiming to make better task allocation decisions. The adaptable task allocation method is an auxiliary mechanism which can be used to enhanced, not replaced, the original EFT mechanism in HEFT [1] or the OEFT mechanism in PEFT [2].

Figures 3.30 and 3.31 show an illustrative example comparing HEFT [1] and the adaptable task allocation method. When making the allocation decision of task 10, processors 1 and 2 look the same regarding its earliest finish time. Therefore, HEFT chooses the first qualified processor, i.e. processors 1, for allocation. On the other hand, our adaptable task allocation method allocates task 10 onto processor 2 because that could allow task 4 to be allocated onto processor 1, resulting in its earlier start time and thus a shorter makespan of the entire schedule than HEFT.

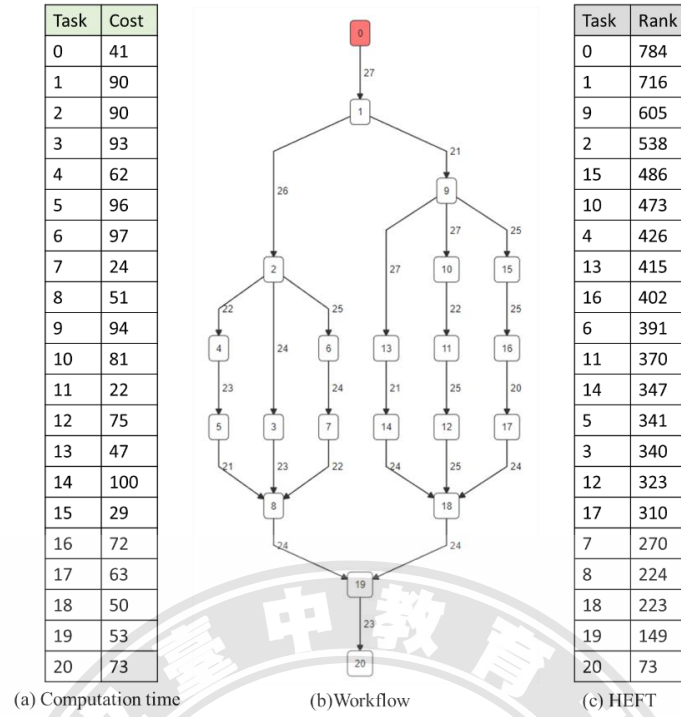


Figure 3.30: CCR 1 workflow scheduling ranking phase

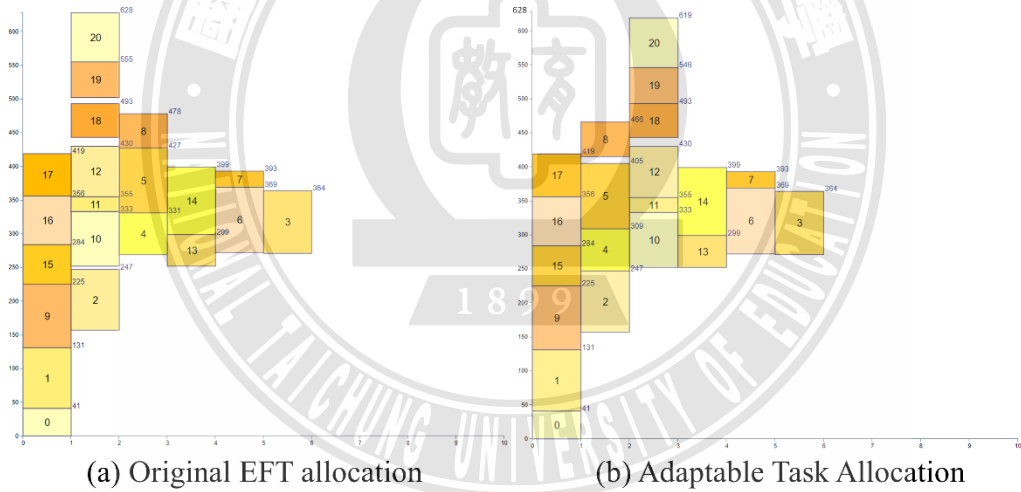


Figure 3.31: HEFT allocation phase

Figures 3.32 and 3.33 presents another example comparing PEFT [2] and our adaptable task allocation method. In this example, our approach allocates task 17 onto processor 2 instead of processor 1 in PEFT's schedule. Such allocation allows task 7 to be allocated on processor 1 and start earlier, resulting a shorter makespan that PEFT [2].

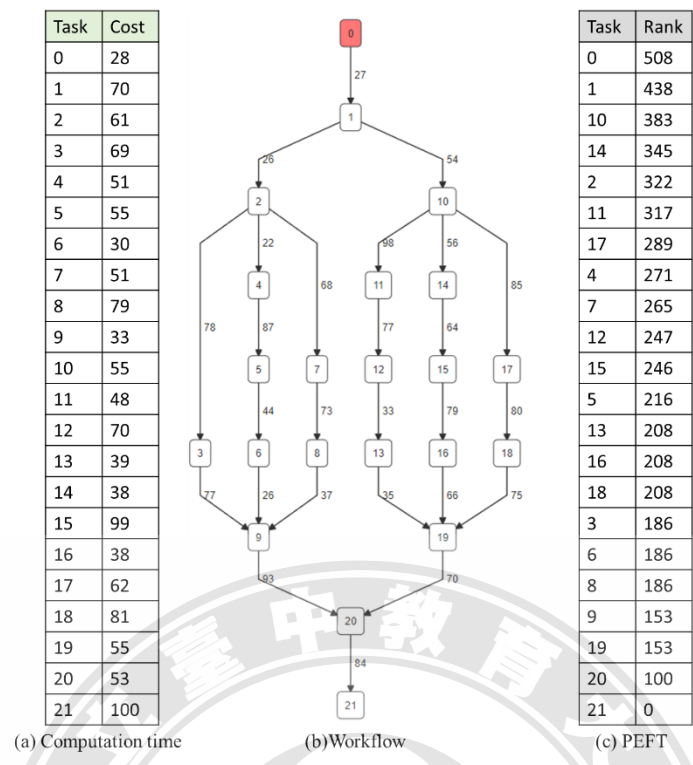


Figure 3.32: CCR 1 workflow scheduling ranking phase

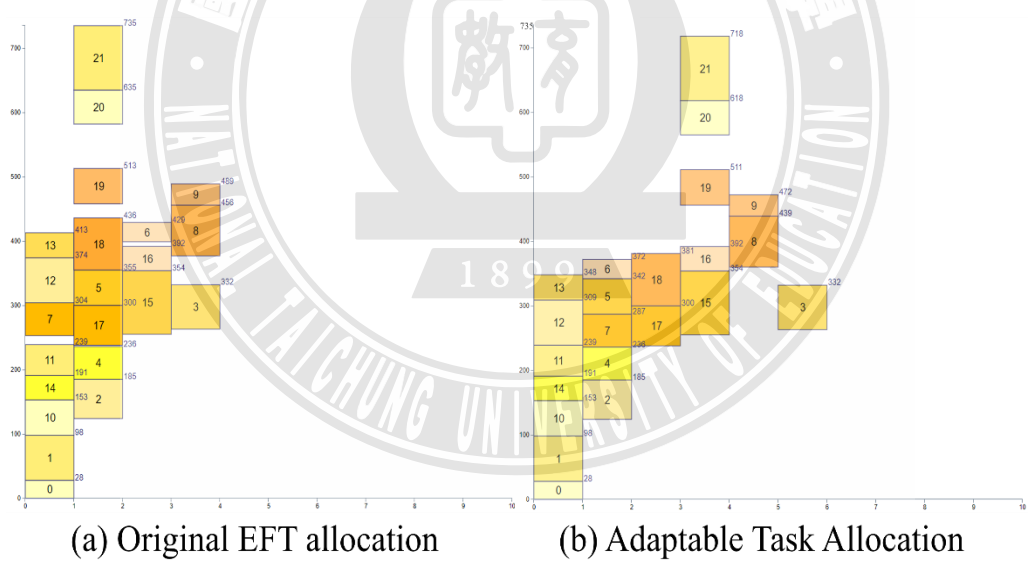


Figure 3.33: PEFT allocation phase

3.2.4 The Entire Workload- and Resource-Aware List

Scheduling Approach

Based on the above three innovative task ranking and allocation mechanisms, the new workload- and resource-aware list-based workflow scheduling approach is described in Algorithm 3.1. Figures 3.34 and 3.35 present an illustrative example comparing HEFT [1] and our approach, and Figures 3.36 and 3.37 show another example comparing PEFT [2] and our approach. The two examples demonstrate that our approach has potential to outperform the two important previous methods in terms of makespan.

Algorithm 3.1: Workload- and resource-aware list scheduling

- 1: Compute the summation of allocated top rank (formula (3.7)) and structural task ranking (formula (3.4)) for all tasks
 - 2: Create an empty ready-list and insert t_{entry} as the initial task
 - 3: **While** ready-list is not empty do
 - 4: $t_i \leftarrow$ the task with the highest rank from ready-list
 - 5: **for all** processor p_k in the system
 - 6: Compute $EFT(t_i, p_k)$ using the insertion-based method
 - 7: **end for**
 - 8: **if** multiple processors having the same minimum EFT value
 - 9: $p_f \leftarrow$ one of the processors minimizing EFT of task t_i and having no precedent tasks
 - 10: **end if**
 - 11: Assign task t_i to processor p_f
 - 12: Re-compute the summation of allocated top rank and structural task ranking for all tasks
 - 13: Update ready-list with new ready tasks
 - 14: **End while**
-

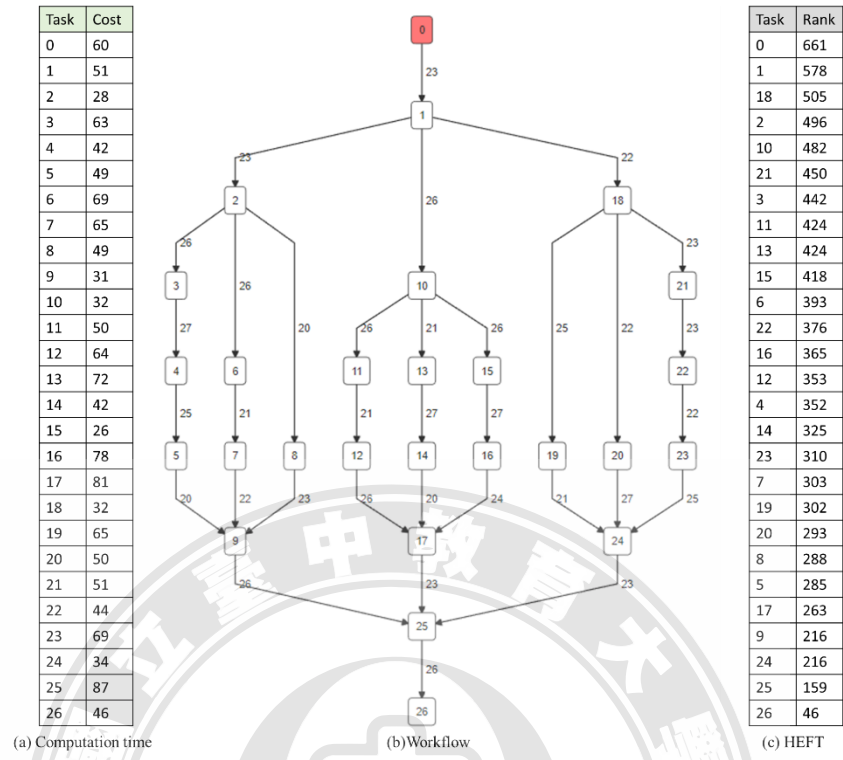


Figure 3.34: CCR 0.1 workflow scheduling ranking phase

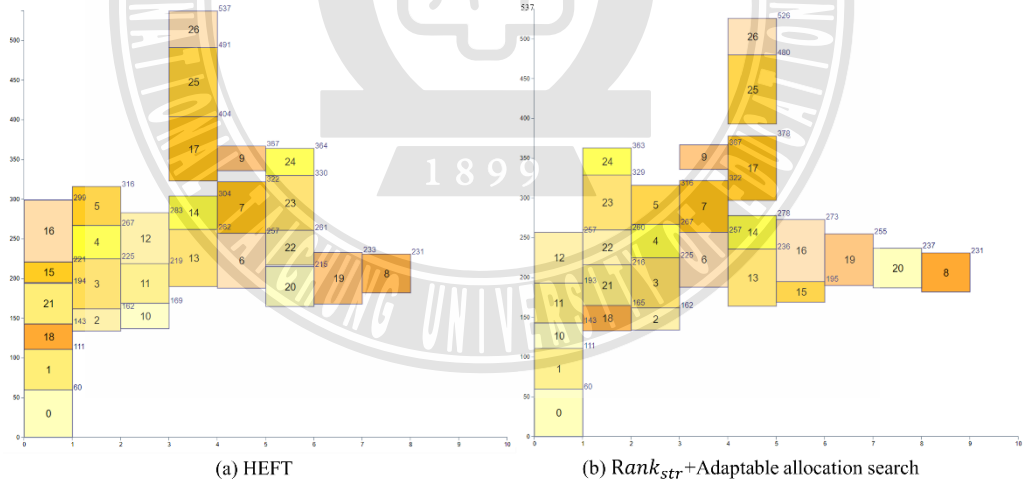
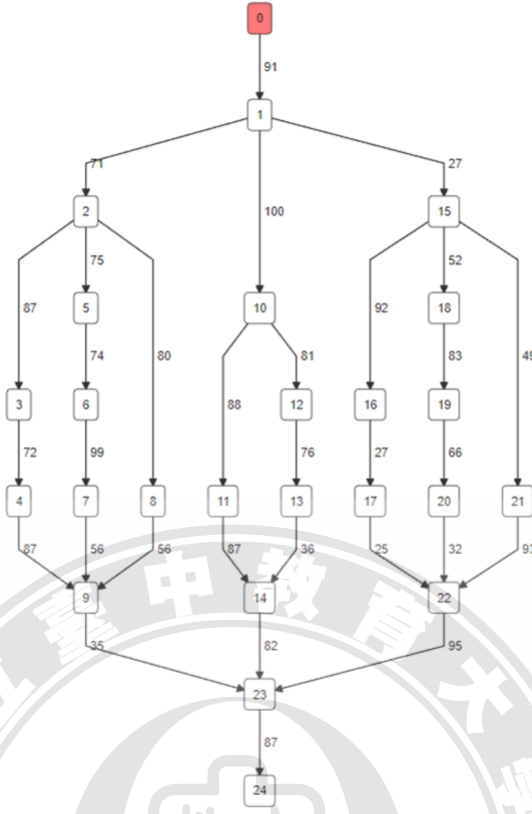


Figure 3.35: CCR 0.1 workflow allocation phase on 10 processors

Task	Cost
0	40
1	50
2	98
3	96
4	46
5	79
6	80
7	77
8	81
9	88
10	82
11	26
12	74
13	67
14	37
15	49
16	89
17	49
18	87
19	97
20	67
21	100
22	73
23	35
24	67

(a) Computation time



(b) Workflow

Task	Rank
0	574
1	524
2	426
15	426
5	347
18	339
10	280
6	267
19	242
3	236
16	224
12	206
4	190
7	190
8	190
17	175
20	175
21	175
11	139
13	139
9	102
14	102
22	102
23	67
24	0

(c) PEFT

Figure 3.36: CCR 1 workflow scheduling ranking phase

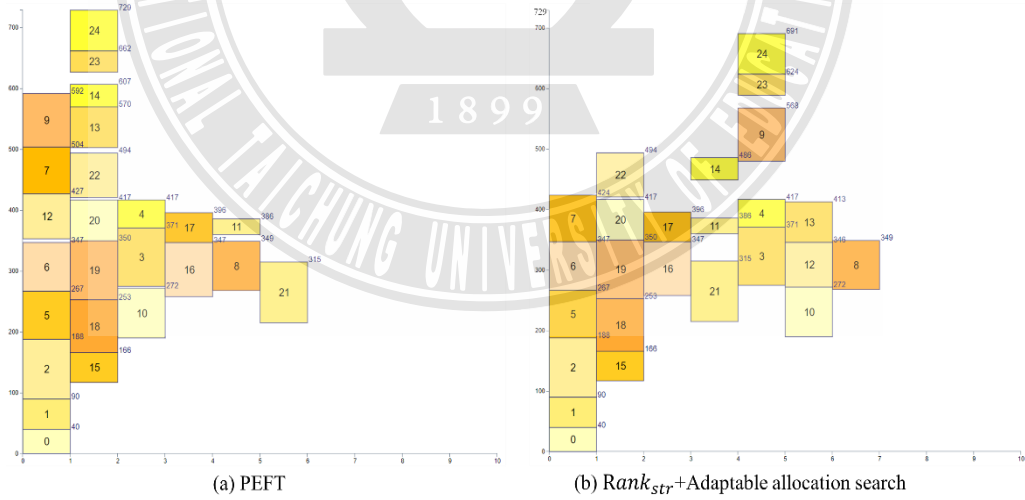


Figure 3.37: CCR 0.1 workflow allocation phase on 10 processors

Chapter 4. Performance Evaluation

This chapter describes the performance evaluation of the new workload- and resource-aware list-based workflow scheduling approach, compared to the two important previous methods, i.e. HEFT [1] and PEFT [2]. The performance evaluation was conducted using a series of simulation experiments configured with different workflow properties and numbers of processors. Two performance metrics were used in comparison of different scheduling methods, average makespan and best ratio. The best ratio measures the number of workflows achieving the best performance when scheduled with a specific algorithm. When more than one algorithm achieves the best performance for a workflow, all the algorithms count it.

Section 4.1 presents experiments using synthetic fork-join workflows, and section 4.2 shows the experimental results based on real-world workflow structures, including LIGO [43][44], Montage [45], and CyberShake [46]. Each experiment involves 1000 workflows and the average performance of them were measured.

4.1 Experiments with Synthetic Fork-Join Workflows

Figure 4.1 presents the experimental results based on synthetic workflows of fork-join structure. Each figure shows a set of experiments running on a specific number of processors involved in workflows with different CCR values. It is obvious that our approach has a larger advantage for workflows of high CCR values. Figure 4.2 compares these scheduling methods in terms of best ratio. The experimental results clearly indicate that when workflows are running on more processors, i.e. 10 processors, our approach outperforms HEFT [1] and PEFT [2] across all CCR values. However, our approach loses its advantage in cases of fewer processors, i.e. 3 processors, especially for workflows of smaller CCR values.

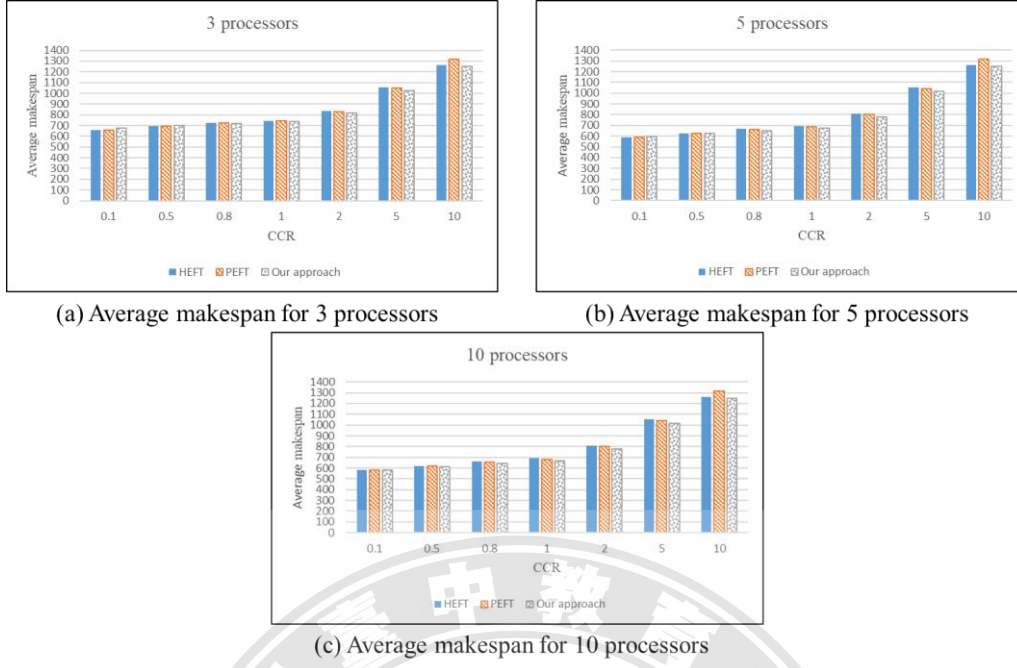


Figure 4.1: Average makespan for different processors

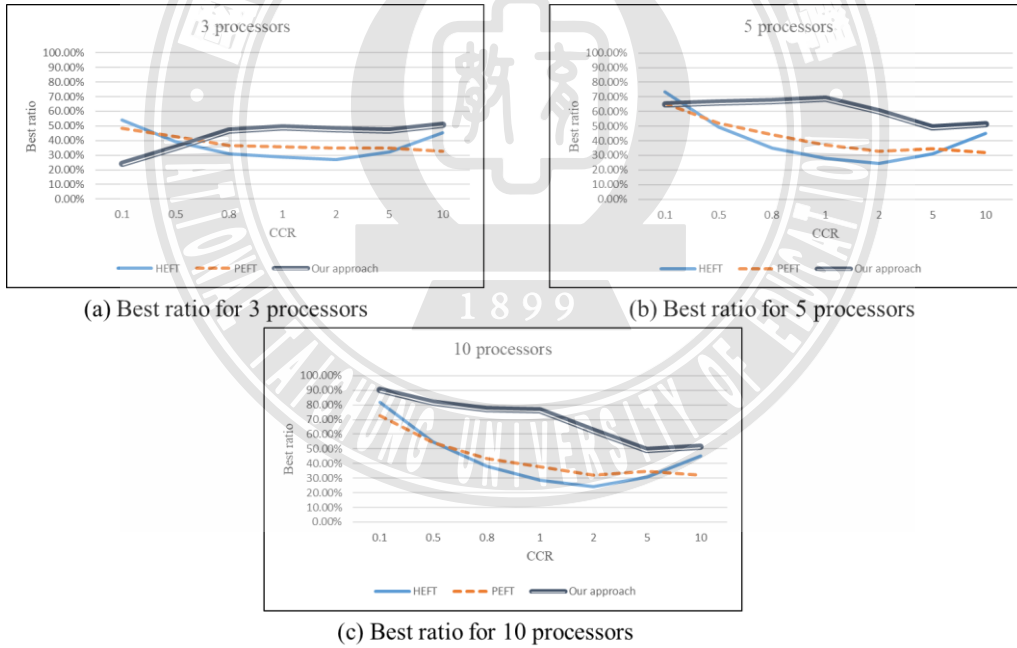


Figure 4.2: Best ratio for different processors

Figures 4.3 and 4.4 present experimental results in another perspective where each figure focuses on experiments with workflows of the same CCR value. The experimental results indicate that our approach outperforms HEFT [1] and PEFT [2] across different numbers of processors for workflows of high CCR values, i.e. 10. For

workflows of small CCR values, i.e. 0.1, our approach has advantage only when processors are enough relative to the workload.

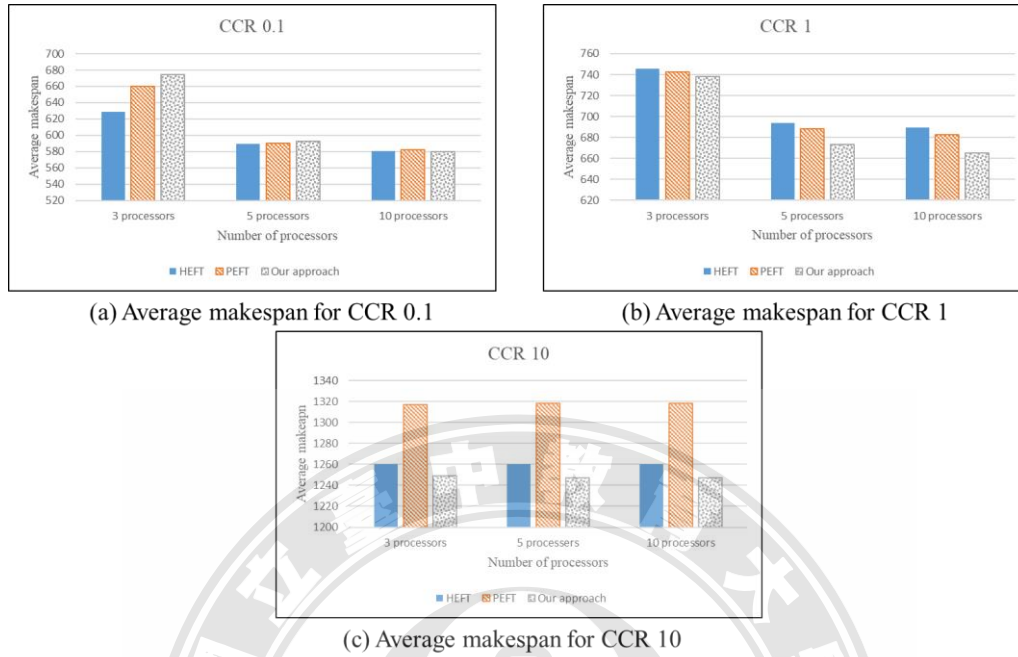


Figure 4.3: Average makespan for CCR value

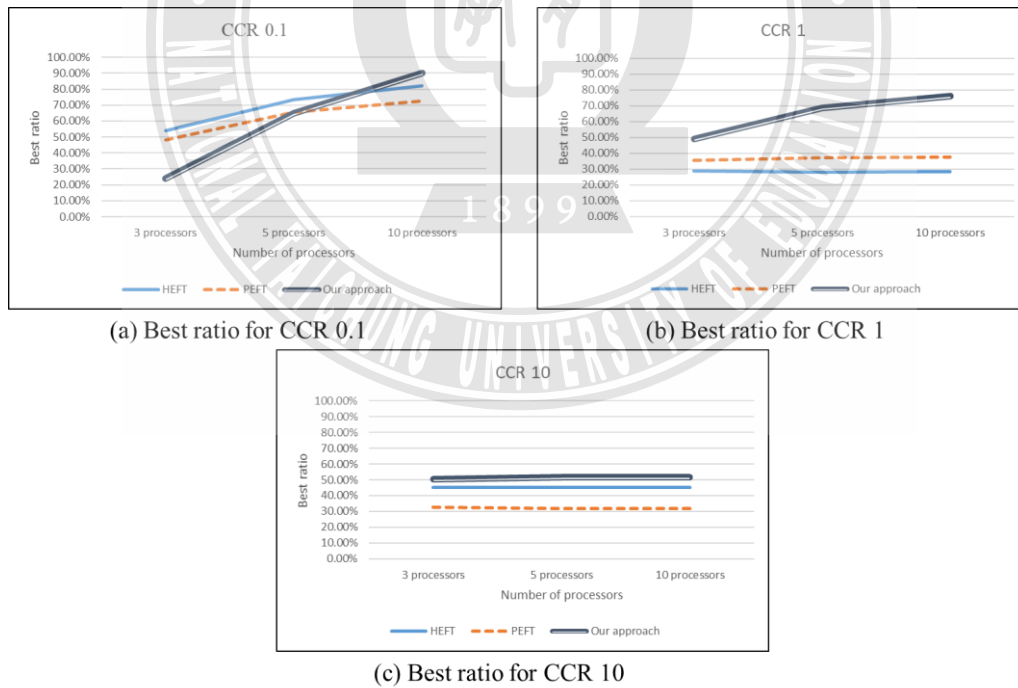
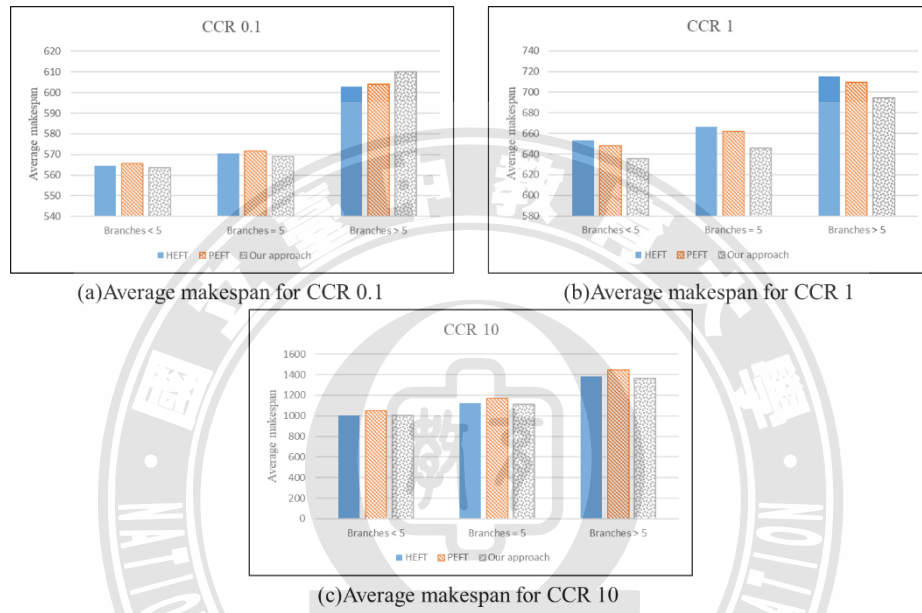


Figure 4.4: Best ratio for different CCR value

Figures 4.5 and 4.6 compare the scheduling methods considering the relationship between workload and resources. For workflows of large and medium CCR values, our

approach is superior to HEFT [1] and PEFT [2] for all types of workload, i.e. different numbers of branches. However, for workflows of small CCR values, i.e. 0.1, our approach loses its advantage for high workload, i.e. large number of branches. Figures 4.7 and 4.8 present experimental results in another perspective. It is clear that our approach achieves superior performance for workflows of smaller branches or larger CCR values.



Figures 4.5: Average makespan with 5 processors for different CCR values

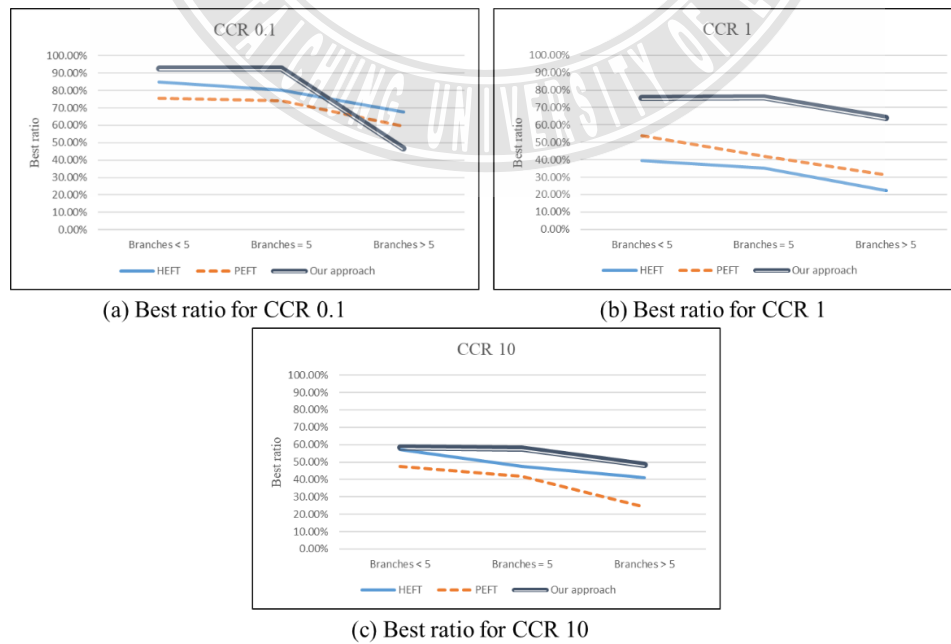


Figure 4.6: Best ratio with 5 processors for different CCR values

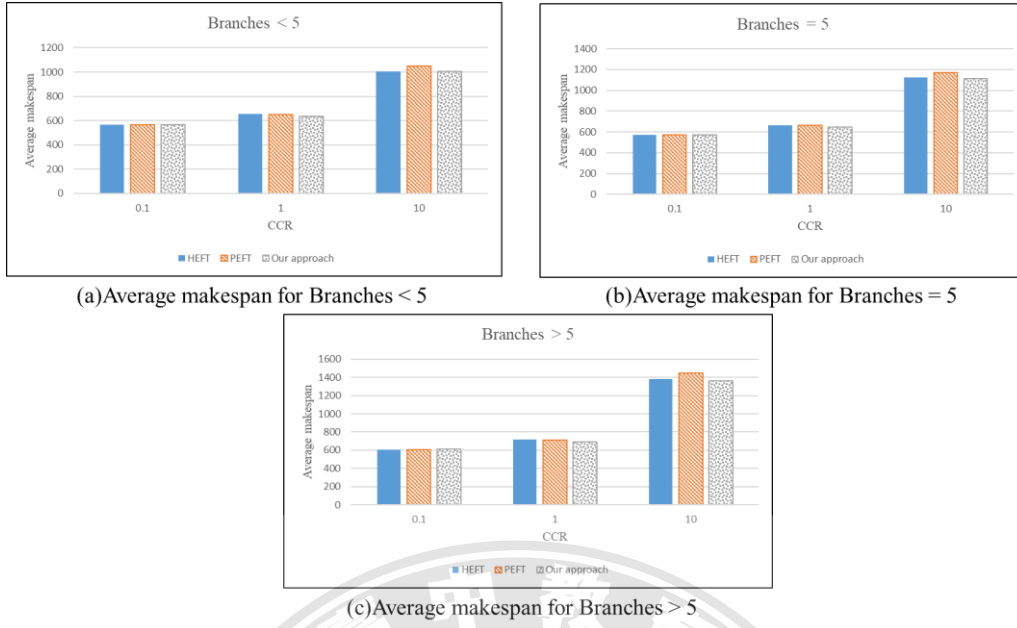


Figure 4.7: Average makespan with 5 processors for different numbers of branches

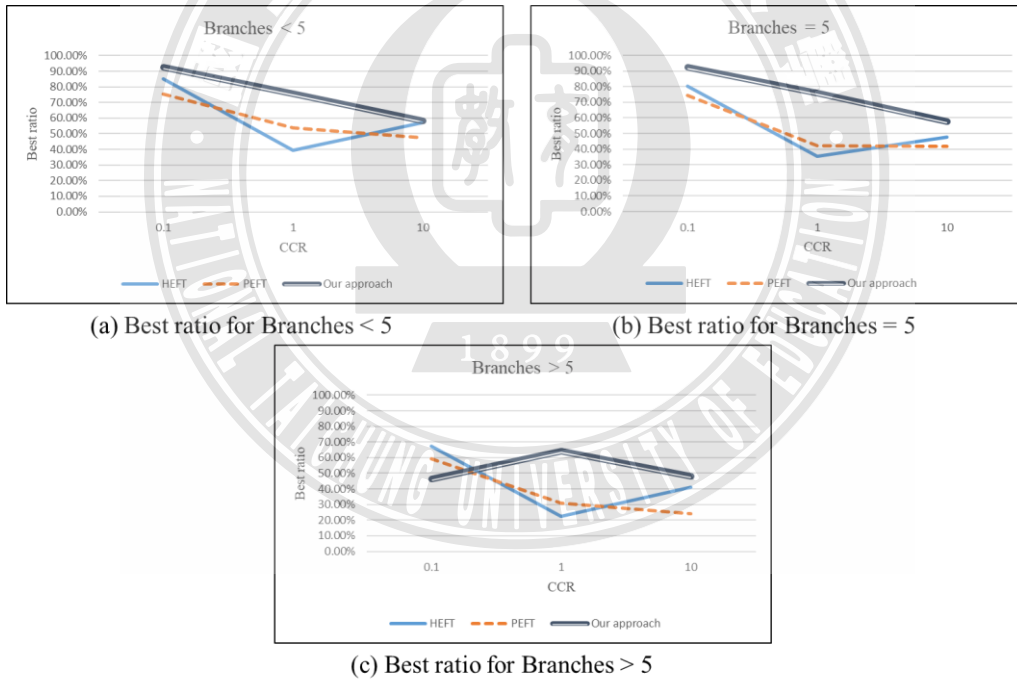


Figure 4.8: Best ratio with 5 processors for different numbers of branches

4.2 Experiments with Real-World Workflow Models

This section presents experimental results based on several real-world workflow models [46].

4.2.1 LIGO

Laser Interferometer Gravitational Wave Observatory (LIGO) [43] is a network of gravitational-wave detectors. It becomes a source of gravitational waves in the universe such as neutron stars and black holes. The DAG diagram of the LIGO workflow is shown in Figure 4.9. Figure 4.10 and Table 4.1 compare the scheduling methods in terms of makespan and best ratio, respectively. Our approach achieves significantly better performance than HEFT [1] and PEFT [2], especially when best ratio is concerned.

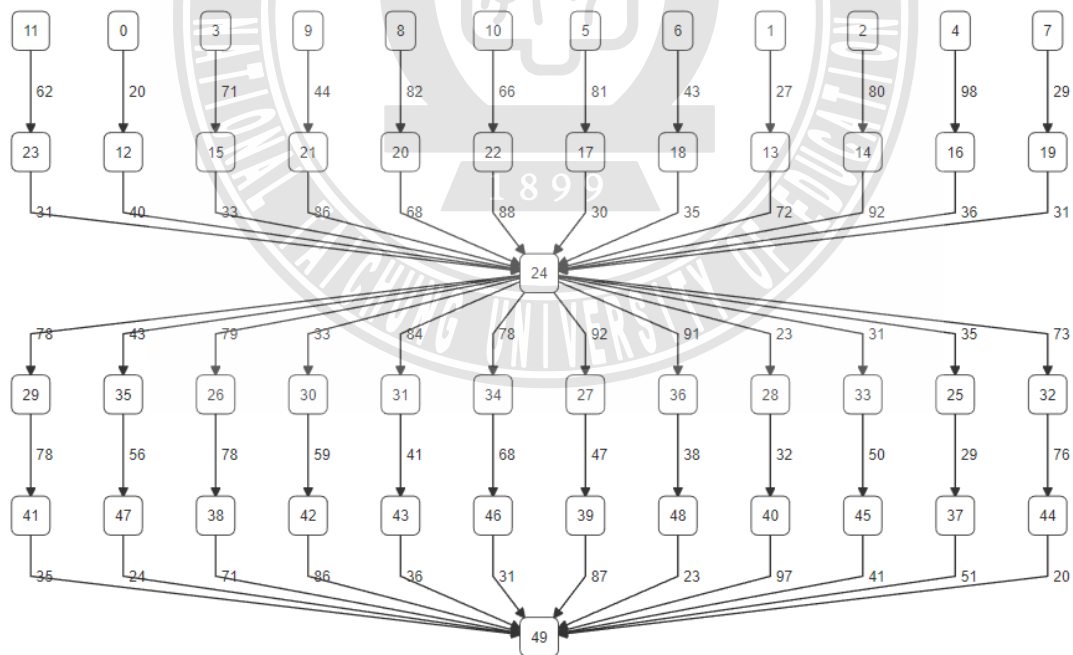


Figure 4.9: Visualized LIGO workflow

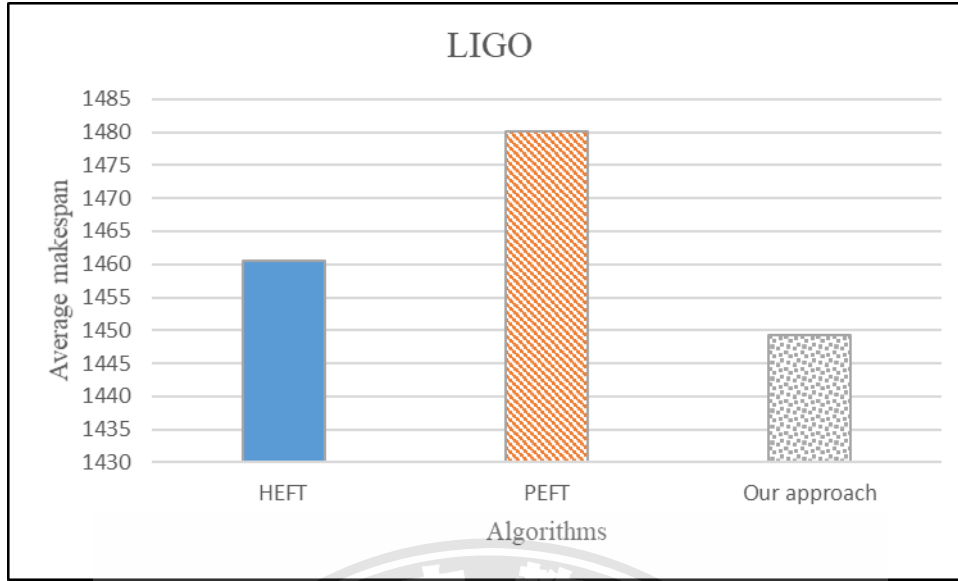


Figure 4.10: Average makespan for LIGO workflow on 10 processors

Table 4.1: Best ratio for LIGO workflow on 10 processors

	HEFT	PEFT	Our approach
Best ratio	22.0%	11.1%	85.5%

4.2.2 Montage

Montage was built by NASA [44], delivering science-grade mosaics of the sky. It has been used in large scientific space telescopes to generate mosaic images. The DAG diagram of the Montage workflow is shown in Figure 4.11. Figure 4.12 and Table 4.2 show that HEFT [1] achieves the best performance in terms of both performance metrics. The main reason for the worse performance of our approach is that inter-task communication in Montage might span across several layers, resulting in inaccurate estimation of communication costs in our approach.

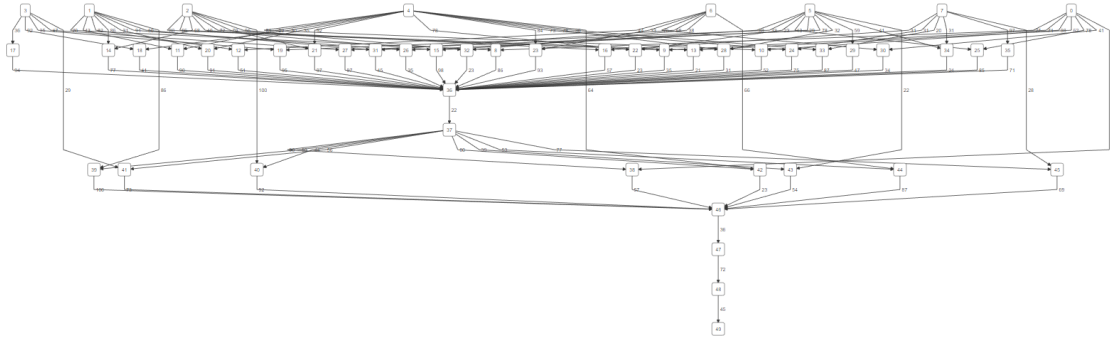


Figure 4.11: Visualized Montage workflow

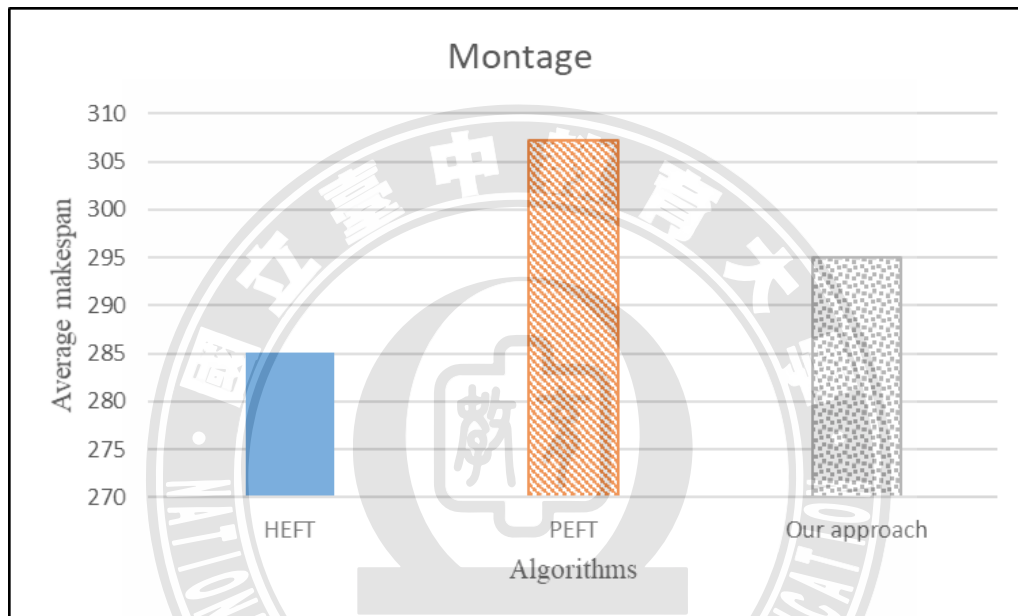


Figure 4.12: Average makespan for Montage workflow on 10 processors

Table 4.2 Best ratio for Montage workflow on 10 processors

	HEFT	PEFT	Our approach
Best ratio	84.6%	21.8%	44.7%

4.2.3 CyberShake

The Southern California Earthquake Center uses the CyberShake workflow [29] to describe earthquake disasters in a region. It can analyze earthquakes and prevent disasters. The DAG diagram of the CyberShake workflow is shown in Figure 4.13. Figure 4.14 and Table 4.3 indicate that our approach outperforms both HEFT [1] and PEFT [2] for CyberShake workflow.

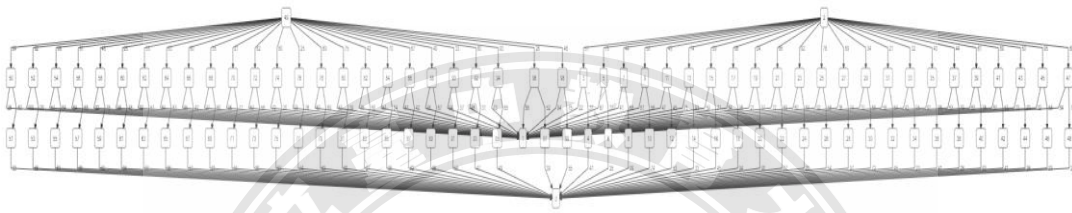


Figure 4.13: Visualized CyberShake workflow

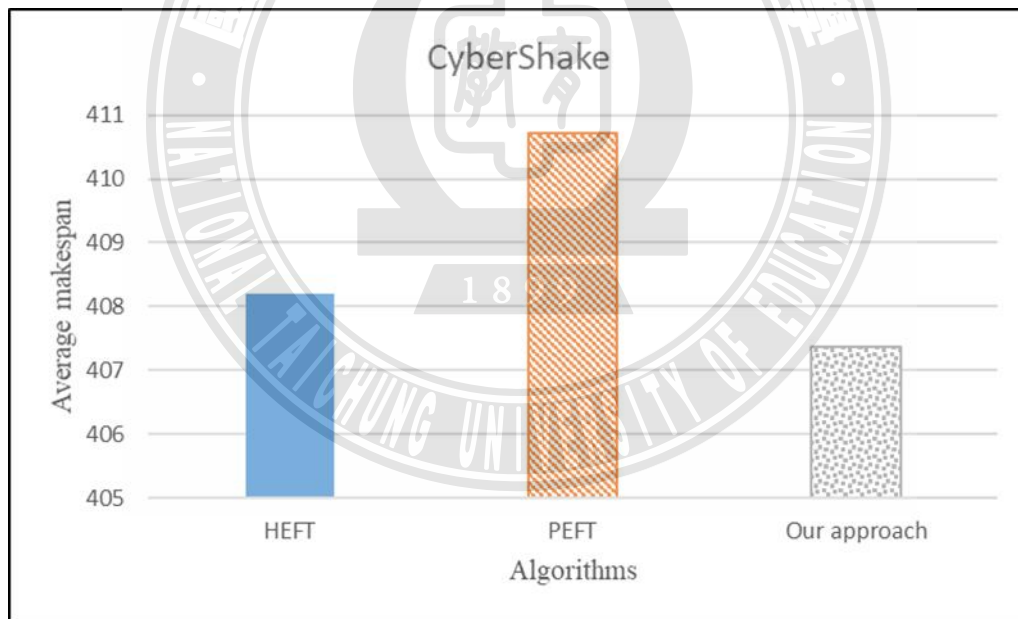


Figure 4.14: Average makespan for CyberShake workflow on 10 processors

Table 4.3: Best ratio for CyberShake workflow on 10 processors

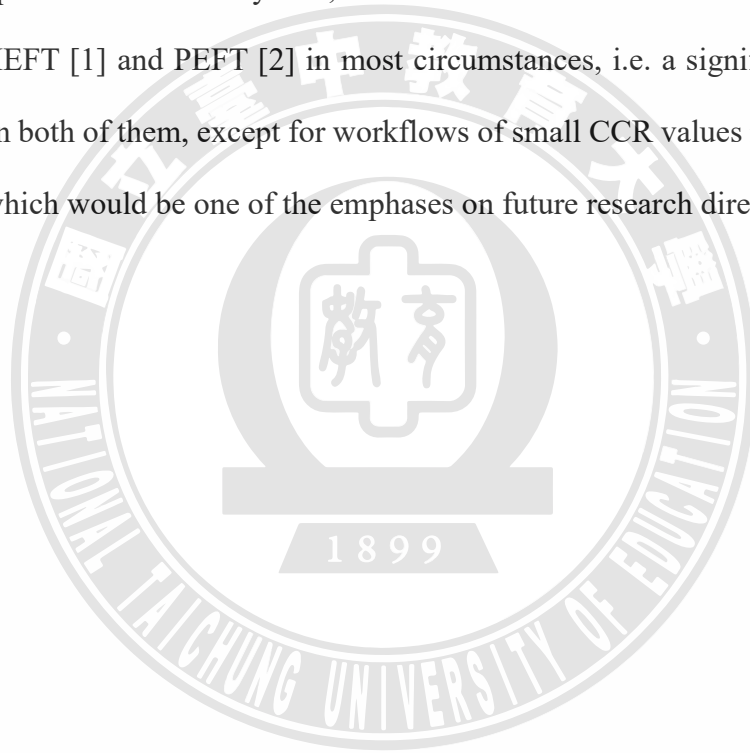
	HEFT	PEFT	Our approach
Best ratio	72.2%	53.9%	79.3%

Chapter 5. Conclusion

In this thesis, we propose and evaluate a new workload- and resource-aware list-based scheduling approach for task-parallel workflows running on homogeneous parallel systems. Speed-homogeneous parallel computing environments are becoming widely available on modern cloud computing platforms with the help of virtualization technology. Traditional well-known list-based scheduling algorithms, e.g. HEFT [1] and PEFT [2], aimed to deal with the workflow scheduling problem on heterogeneous computing systems, and thus confronted the issue of inaccurate computation cost estimation, affecting the resultant schedule quality greatly. In this thesis, by focusing on homogeneous parallel systems, more intrinsic properties and issues in workflow scheduling can be investigated and thus scheduling algorithms can be improved.

The proposed workload- and resource-aware list-based workflow scheduling approach consists of three innovative mechanisms in task ranking and allocation, including structural task ranking, task ranking based on allocated critical path, and adaptable task allocation. The structural task ranking mechanism can make more accurate communication cost estimation by analyzing inter-task relationship, and thus determine a more appropriate task allocation sequence to effectively reduce workflow execution makespan. In addition, our task ranking mechanism is based on allocated critical path instead of the bottom rank used in HEFT [1] and PEFT [2], improving workflow execution makespan by reducing the execution and communication length along the critical path. Finally, in the task allocation stage, in contrast to considering only each task itself, e.g. HEFT [1], or each task's successors, e.g. PEFT [2], our adaptable method also takes into consideration non-successor tasks, and thus is promising to make better allocation decisions for effectively reducing workflow execution makespan.

The proposed approach was evaluated with a series of simulation experiments using both synthetic workflow structures and real-world workflow models, and compared to HEFT [1] and PEFT [2] in terms of both average makespan and best ratio. Average makespan is the widely used performance metric from the system perspective used in most previous workflow scheduling research works, while best ratio is more likely to be the concern of individual users. The experimental results were discussed and analyzed in different perspectives, e.g. various numbers of branches in workflows, numbers of processors in the system, and different CCR values. Our approach is superior to HEFT [1] and PEFT [2] in most circumstances, i.e. a significantly higher best ratio than both of them, except for workflows of small CCR values running in few processors, which would be one of the emphases on future research directions.



References

- [1] Topcuoglu, H., Hariri, S., & Wu, M.Y. (2002). Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE Transactions on Parallel and Distributed Systems*, 13(3), 260-274. doi: 10.1109/71.993206
- [2] Arabnejad, H., & Barbosa J. G. (2013). List scheduling algorithm for heterogeneous systems by an optimistic cost table. *IEEE Transactions On Distributed Systems*, 25(3), 682-694. doi: 10.1109/TPDS.2013.57
- [3] Bittencourt, L. F., Sakellariou, R., & Madeira, E. R. M. (2010). Dag scheduling using a lookahead variant of the heterogeneous earliest finish time algorithm. *18th Euromicro Conference on Parallel, Distributed and Network-based Processing*, 27-34. doi: 10.1109/PDP.2010.56
- [4] Abazari, F., Analoui, M., Takabi, H., & Fu, S. (2019). MOWS: Multi-objective workflow scheduling in cloud computing based on heuristic algorithm. *Simulation Modelling Practice and Theory*, 93, 119-132. doi: 10.1016/j.simpat.2018.10.004
- [5] Khan, M. A. (2012). Scheduling for heterogeneous systems using constrained critical paths. *Parallel Computing*, 38(4-5), 175-193. doi: 10.1016/j.parco.2012.01.001
- [6] Cao, T. M. Z., Ye, H. Z., & Hu, Z. D. (2018). k-HEFT: A static task scheduling algorithm in clouds. *ACM-ICPS ACM International Conference Proceeding Series*, 152-159. doi: 10.1145/3232116.3232141
- [7] Ijaz, S. & Munir, E. U. (2018). MOPT: list-based heuristic for scheduling workflows in cloud environment. *The Journal of Supercomputing*, 1-29. doi: 10.1007/s11227-018-2726-6
- [8] Ahmad, W., Alam, B., & Malik, S. (2019). Performance Analysis of List Scheduling Algorithms by Random Synthetic DAGs. *Proceedings of 2nd International Conference on Advanced Computing and Software Engineering (ICACSE)*. doi: 10.2139/ssrn.3349016
- [9] Khan, M. A. (2018). Scheduling Heterogeneous Systems Using Relative Latencies. *Arabian Journal for Science and Engineering*, 43(2), 569-584. doi: 10.1007/s13369-017-2689-6
- [10] Witt, C., Wheating, S., & Leser, U. (2019). LOS: Level Order Sampling for Task Graph Scheduling on Heterogeneous Resources. *IEEE/ACM Workflows in Support of Large-Scale Science (WORKS)*. doi: 10.1109/WORKS.2018.00008
- [11] Atef, A., Hagrass, T., Mahdy, Y. B., & Janeček, J. (2017). Lower-bound complexity algorithm for task scheduling on heterogeneous grid. *Computing*, 99(11), 1125-1145. doi: 10.1007/s00607-017-0558-5
- [12] AlEbrahim, S. & Ahmad, I. (2017). Task scheduling for heterogeneous computing

- systems. *The Journal of Supercomputing*, 73(6), 2313-2338. doi: 10.1007/s11227-016-1917-2
- [13] Zhou, N., Qi, D., Wang, X., Zheng, Z., & Lin, W. (2017). A list scheduling algorithm for heterogeneous systems based on a critical node cost table and pessimistic cost table. *Concurrency and computation practice and experience*, 29(5), 2017. doi: 10.1002/cpe.3944
- [14] Huang, K. C. & Tsai, M. H. (2016). Task ranking and allocation heuristics for efficient workflow schedules. *International Computer Symposium (ICS)*. doi: 10.1109/ICS.2016.0108
- [15] Wang, G., Guo, H., & Wang, Y. (2015). A novel heterogeneous scheduling algorithm with improved task priority. *IEEE 17th International Conference on High Performance Computing and Communications*. doi: 10.1109/HPCC-CSS-ICISS.2015.48
- [16] Huang, K. C., Tsai, Y. L., & Liu, H. C. (2015). Task ranking and allocation in list-based workflow scheduling on parallel computing platform. *The Journal of Supercomputing*, 71(1), 217-240. doi: 10.1007/s11227-014-1294-7
- [17] Bittencourt, L. F. & Madeira, E. R. M. (2010). Towards the scheduling of multiple workflows on computational grids. *Journal of Grid Computing*, 8(3), 419-441. doi: 10.1007/s10723-009-9144-1
- [18] Liou, J & Palis, M. A. (1996). An efficient clustering heuristic for scheduling dags on multiprocessors. *Proceedings of the 8th symposium on parallel and distributed processing*.
- [19] Boeres, C., Filho, J. V., & Rebello, V. E. F. (2004). A cluster-based strategy for scheduling task on heterogeneous processors. *16th Symposium on Computer Architecture and High Performance Computing*, 214-221. doi: 10.1109/SBAC-PAD.2004.1
- [20] Dai, Y. & Zhang, X. (2014). A synthesized heuristic task scheduling algorithm. *The Scientific World Journal*, 2014(1), 1–9. doi: 10.1155/2014/465702
- [21] Tang, X., Li, K., Liao, G., & Li, R. (2010). List scheduling with duplication for heterogeneous computing systems. *Journal of Parallel and Distributed Computing*, 70(4), 323–329.
- [22] Correa, R. C., Ferreira, A., & Rebreyend, P. (1999). Scheduling multiprocessor tasks with genetic algorithms. *IEEE Transactions on Parallel and Distributed Systems*, 10(8), 825–837. doi: 10.1109/71.790600
- [23] Maurya, A. K. & Tripathi, A. K. (2017). Performance Comparison of HEFT, Lookahead, CEFT and PEFT Scheduling Algorithms for Heterogeneous Computing Systems. *ACM-ICPSACM International Conference Proceeding Series*, 128-132. doi: 10.1145/3154979.3154985

- [24] Ullman, J. D. (1975). NP-complete scheduling problems. *Journal of Computer and System sciences*, 10(3), 384-393. doi: 10.1016/S0022-0000(75)80008-0
- [25] Pinedo, M. L. (2012). *Scheduling: Theory, Algorithms, and Systems* (4th ed.). New York, NY: Springer-Verlag. doi: 10.1007/978-1-4614-2361-4
- [26] Sinnen, O. (2006). *Task Scheduling for Parallel Systems*. New York, NY: John Wiley & Sons, Inc.
- [27] Deelman, E., Singh, G., Su, M. H., Blythe, J., Gil, Y., Kesselman, C., Mehta, G., Vahi, K., Berriman, G. B., Good, J., Laity, A., Jacob, J. C., & Katz D. S. (2005). Pegasus: A framework for mapping complex scientific workflows onto distributed systems. *Scientific Programming*, 13(3), 219–237. doi: 10.1155/2005/128026
- [28] Ramakrishnan. A., Singh, G., Zhao, H., Deelman, E., Sakellariou, R., Vahi, K., Blackburn, K., Meyers, D., & Samidi, M. (2007). Scheduling data-intensive workflows onto storage-constrained distributed resources. *Seventh IEEE International Symposium on Cluster Computing and the Grid*, 401–409. doi: 10.1109/CCGRID.2007.101
- [29] Maechling, P., Deelman, E., Zhao, L., Graves, R., Mehta, G., Gupta, N., J., Kesselman, C., Callaghan, S., Okaya, D., Francoeur, H., Gupta, V., Cui, Y., Vahi, K., Jordan, T., & Field, E. (2007). SCEC CyberShake Workflows-Automating Probabilistic Seismic Hazard Analysis Calculations. *Workflows for e-Science*, 143-163. doi: 10.1007/978-1-84628-757-2_10
- [30] Ibrahim S., He, B., & Jin, H. (2011). Towards Pay-As-You-Consume Cloud Computing. *IEEE International Conference on Services Computing*, 370-377. doi: 10.1109/SCC.2011.38
- [31] Hsu, C. C., Huang, K. C., & Wang, F. J. (2010). Online Scheduling of Workflow Applications in Grid Environment. *International Conference on Grid and Pervasive Computing*, 300-310. doi: 10.1007/978-3-642-13067-0_33
- [32] Wong, A. K. & Goscinski, A. M. (2008). The impact of under-estimated length of jobs on easy-backfill scheduling. *16th Euromicro Conference on Parallel, Distributed and Network-Based Processing*, 343-350. doi: 10.1109/PDP.2008.53
- [33] Wu. X. & Loiseau, P. (2015). Algorithms for Scheduling Deadline-Sensitive Malleable Tasks. *53rd Annual Allerton Conference on Communication, Control, and Computing*, 530-537. doi: 10.1109/ALLERTON.2015.7447050
- [34] Huang, K. C., Wu, W. Y., Wang, F. J., Liu, H. C., & Hung, C. H. (2016). An Iterative expanding and shrinking process for processor allocation in mixed-parallel workflow scheduling. *SpringerPlus*, 860–870. doi: 10.1186/s40064-016-2808-y
- [35] Benoit, A., Çatalyürek, Ü. V., Robert, Y., & Saule, E. (2013). A survey of pipelined workflow scheduling: Models and algorithms. *ACM Computing Surveys*, 45(4). doi: 10.1145/2501654.2501664

- [36] Ahmad, S. G., Liew, C. S., Rafique, M. M., & Munir, E. U. (2017). Optimization of data-intensive workflows in stream-based data processing models. *The Journal of Supercomputing*, 73(9), 3901-3923. doi: 10.1007/s11227-017-1991-0
- [37] Wang, Y. R., Huang, K. C., & F. J. Wang. (2016). Scheduling online mixed-parallel workflows of rigid tasks in heterogeneous multi-cluster environments. *Future Generation Computer Systems*, 60, 35-47. doi: 10.1016/j.future.2016.01.013
- [38] Liu, J., Ren, J., Dai, W., Zhang, D., Zhou, P., Zhang, Y., Min, G., & Najjari, N. (2019). Online Multi-Workflow Scheduling under Uncertain Task Execution Time in IaaS Clouds. *IEEE Transactions on Cloud Computing*. doi: 10.1109/TCC.2019.2906300
- [39] Li, D. & Wu, J. (2013). Energy-Aware Scheduling for Acyclic Synchronous Data Flows on Multiprocessors. *Journal of Interconnection Networks*, 14(3). doi: 10.1142/S0219265913500114
- [40] Baskiyar, S. & Abdel-Kader, R. (2010). Energy aware DAG scheduling on heterogeneous systems. *Cluster Computing*, 13(4), 373-383. doi: 10.1007/s10586-009-0119-6
- [41] Alkhanak, E. N., Lee, S. P., & Khan, S. U. R. (2015). Cost-aware challenges for workflow scheduling approaches in cloud computing environments: Taxonomy and opportunities. *Future Generation Computer Systems*, 50, 3-21. doi: 10.1016/j.future.2015.01.007
- [42] Arabnejad, V. & Bubendorfer, K. (2015). Cost Effective and Deadline Constrained Scientific Workflow Scheduling for Commercial Clouds. *IEEE 14th International Symposium on Network Computing and Applications*. 106-113. doi: 10.1109/NCA.2015.33
- [43] Abramovici, A., Althouse W. E., Drever, R. W. P., R., Gürsel, Y., Kawamura, S., Raab, F. J., Shoemaker, D., Sievers, L., Spero, R. E., Thorne, K. S., Vogt, R. E., Weiss, R., Whitcomb, S. E., & Zucker, M. E. (1992). LIGO: The laser interferometer gravitational-wave observatory. *Science*, vol. 256(5055), 325-333. doi: 10.1126/science.256.5055.325
- [44] Deelman, E., Singh, G., Livny, M., Berriman, B., & Good J. (2008) The cost of doing science on the cloud: The Montage example. *ACM/IEEE Conference on Supercomputing*, 1-12. doi. 10.1109/SC.2008.5217932
- [46] SyntheticWorkflows. (2017). Retrieved from: <https://pegasus.isi.edu/>